VIETNAM NATIONAL UNIVERITY, HO CHI MINH CITY

HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

FALCULTY OF ELECTRICAL - ELECTRONIC ENGINEERING

DEPARTMENT OF ELECTRONICS

----------o0o----------

CAPSTONE PROJECT 2

CGRA ACCELERATOR FOR SNN INFERENCE ON FPGA

Instructor: Assoc. Prof, PhD. Truong Quang Vinh

Student name: Huynh Pham Anh Duy – 2051095

Trieu Thanh Vinh – 2213980

Ho Chi Minh City, 30th December 2025

ACKNOWLEDGEMENT

The process of finishing this Capstone Project 2 has been one of progress, obstacles, and priceless assistance. Our sincere appreciation goes out to everyone who has helped with this project.

We would like to thank our project instructor, Assoc. Prof., PhD. Truong Quang Vinh, for his continuous support, knowledge, and encouragement have greatly influenced the course and caliber of this research.

We sincerely appreciate our family's unending love, tolerance, and support. Their steadfast faith has served as a continual source of inspiration. We also want to thank our friends for their support, whose conversations, criticism, and encouragement have been very helpful in improving the concepts and working methods.

Lastly, we would like to thank everyone who helped with this project, whether directly or indirectly. Your help, no matter how big or small, has been greatly appreciated and has been essential to the success of this project.

Ho Chi Minh City, 30th December 2025.

Student

Huynh Pham Anh Duy

Trieu Thanh Vinh

ABSTRACT

This project presents the specification and architectural definition of a Coarse-Grained Reconfigurable Architecture (CGRA) accelerator optimized for sparse Spiking Neural Network (SNN) inference. Modern deep learning workloads—particularly event-driven SNNs—are heavily constrained by the "memory wall" due to irregular data access patterns and high weight sparsity. To address these bottlenecks, the proposed accelerator employs a 4×4 mesh of Processing Elements (PEs) interconnected through a multicast-capable routing fabric, enabling efficient execution of Compressed Sparse Column (CSC) operations.

A distinguishing feature of this architecture is the integration of a dedicated Leaky Integrate-and-Fire (LIF) neuron update unit within each PE, allowing biologically inspired neural dynamics to be computed directly in hardware. The system targets deployment on Xilinx Zynq UltraScale+ XCZU4EV, leveraging the ARM Cortex-A53 Application Processing Unit (APU) for control and orchestration, while delegating high-throughput computation to the FPGA fabric. Communication between the subsystems is implemented through AMBA AXI4 for high-bandwidth DMA transfers and APB for configuration and control.

Key innovations include a double-buffered configuration memory enabling seamless, low-latency context switching, and a deterministic spatio-temporal scheduling mechanism that enhances utilization and predictability. Together, these features form a flexible and efficient hardware platform tailored for real-time, power-efficient SNN inference.

TABLE OF CONTENTS

ACKNOWLEDGEMENT 1
ABSTRACT 2
1. INTRODUCTION 8
1.1. Overview of CGRA and SNN Inference 8
1.2. System Context and Execution Flow 9
1.3. Target Software Environment 9
1.4. Target Workload and Problem Scope 10
1.5. Project Objectives 11
2. LITERATURE REVIEW AND BACKGROUND 12
2.1. Overview of Coarse-Grained Reconfigurable Architectures (CGRAs) 12
2.2. Processing Element (PE) and Mesh Network Architecture 13
2.2.1. Overview of the CGRA Processing Element 13
2.2.2. Mesh Routing Network and Data Movement 13
2.3. Spiking Neural Networks (SNNs) 15
2.3.1. Neuron State, Membrane Potential, and Spike Events 16
2.3.2. Leaky Integrate-and-Fire (LIF) Neuron Model 16
2.3.3. Information Encoding Schemes 17
2.3.4. Training Methodologies for SNNs 18
2.3.5. Computational Advantages over Von Neumann Architectures 19
2.3.6. Energy Efficiency and Event-Driven Processing 19
2.3.7. Challenges in SNN Training and Deployment 20
3. SYSTEM ARCHITECTURE 21
3.1. SoC System Integration Block Diagram 21
3.2. Block Diagram Detail of CGRA Accelerator IP 22
3.3. Processing Element (PE) Tile Architecture 23
3.4. Instruction Set Architecture (ISA) 24
3.4.1. Configuration Frame Format 24
3.4.2. Opcode Summary 25
4. HARDWARE DESIGN 27
4.1. Top Module (cgra_top.sv) 27
4.2. CSR (cgra_apb_csr.sv) 30
4.3. Control Unit (cgra_control_unit.sv) 32
4.4. DMA Engine (cgra_dma_engine.sv) 34
4.5. Processing Element (cgra_pe.sv) 37
4.6. Tile Memory (cgra_tile_memory.sv) 39
4.7. Array PE 4x4 (cgra_array_4x4.sv) 41

4.8. Module Title (cgra_tile.sv) 43
4.9. Router (cgra_router.sv) 45
5. VERIFICATION AND TESTING 47
5.1. Verification Methodology 47
5.2. Golden Model Architecture 47
5.3. Testbench Components and Flow 49
5.4. Verification Test Suites 50
5.5. Verification Results 52
6. IMPLEMENTATION RESULTS 53
7. CONCLUSION AND DEVELOPMENT DIRECTION 56
7.1. Conclusion 56
7.2. Development Direction 56
REFERENCES 58

LIST OF FIGURES

Figure 1: Mesh routing for 4x4 CGRA *14*
Figure 2: Spiking Neural Networks (SNNs) *15*
Figure 3: SoC System Integration Block Diagram *21*
Figure 4: Block Diagram for CGRA Accelerator IP *22*
Figure 5: Block diagram for each PE *23*
Figure 6: Configuration Frame *24*
Figure 7: Top Module CGRA *27*
Figure 8: Module CSR (APB Slave) *30*
Figure 9: Module Control Unit *32*
Figure 10: Module DMA Engine *34*
Figure 11: Module Processing Element *37*
Figure 12: Module Tile Memory *39*
Figure 13: Module Array 4x4 PEs *41*
Figure 14: Module Tile *43*
Figure 15: Module Router *45*
Figure 16: Golden Model Behavioral Logic *48*
Figure 17: Layered Testbench Architecture *49*
Figure 18: Randomized ISA Regression log *51*
Figure 19: Verification Results *52*
Figure 20: Waveform Test of CGRA IP *53*
Figure 21: Block Design in Vivado *53*
Figure 22: Result is shown on Hercules through UART *54*
Figure 23: Resource Utilization *54*
Figure 24: Power Report *54*
Figure 25: Timing Report *55*

LIST OF TABLES

Table 1: Configuration Summary *25*
Table 2: Opcodes of ALU *26*
Table 3: Signals of top module CGRA *29*
Table 4: Signals of CSR Module *31*
Table 5: Signals of Control Unit Module *33*
Table 6: Signals of DMA Engine *35*
Table 7: Signals of Processing Element Module *38*
Table 8: Signals of Tile Memory *40*
Table 9: Signals of Array 4x4 ..... *42*
Table 10: Signals of Tile *44*
Table 11: Signals of Router *45*

LIST OF ABBREVIATIONS

ALU	Arithmetic Logic Unit
AMBA	Advanced Microcontroller Bus Architecture
ANN	Artificial Neural Network
APB	Advanced Peripheral Bus
ASIC	Application-Specific Integrated Circuit
AXI	Advanced eXtensible Interface
BPTT	Backpropagation Through Time
CGRA	Coarse-Grained Reconfigurable Architecture
CPU	Central Processing Unit
CSC	Compressed Sparse Column
CSR	Control and Status Register
DDR	Double Data Rate (Memory)
DMA	Direct Memory Access
DRAM	Dynamic Random Access Memory
DVS	Dynamic Vision Sensor
FIFO	First-In, First-Out
FPGA	Field-Programmable Gate Array
FSM	Finite State Machine
GPU	Graphics Processing Unit
HPS	Hard Processor System
IP	Intellectual Property
LIF	Leaky Integrate-and-Fire
LUT	Lookup Table
MAC	Multiply-Accumulate
NoC	Network on Chip
PE	Processing Element
RAM	Random Access Memory
RF	Register File
SIMD	Single Instruction, Multiple Data
SNN	Spiking Neural Network
SoC	System on Chip
SPM	Scratchpad Memory
SRAM	Static Random Access Memory
STDP	Spike-Timing-Dependent Plasticity
VLIW	Very Long Instruction Word

1. INTRODUCTION

1.1. Overview of CGRA and SNN Inference

This specification presents the architecture and design methodology of a 4×4 fixed-point Coarse-Grained Reconfigurable Array (CGRA) accelerator, developed as a domain-specific hardware engine for sparse Spiking Neural Network (SNN) inference. Unlike conventional neural networks that rely on dense tensor operations, SNNs operate on sparse, event-driven computations dominated by irregular memory access patterns and lightweight neuronal dynamics. These characteristics introduce fundamental inefficiencies in traditional von Neumann processors and GPU-based accelerators, where SIMD utilization drops sharply with sparsity.

The proposed CGRA addresses these limitations through a spatially programmable 4×4 mesh of Processing Elements (PEs), each combining a fixed-point Multiply–Accumulate (MAC) unit with a hardware-implemented Leaky-Integrate-and-Fire (LIF) neuron update engine. This unified MAC+LIF execution model enables the array to compute sparse matrix–vector (matvec) operations and neuron dynamics in a tightly coupled, dataflow-driven manner. Execution is governed by cycle-accurate configuration words stored in per-PE context memories, enabling temporal scheduling similar to VLIW-style instruction flows while maintaining deterministic timing.

The accelerator is deployed within the Programmable Logic (PL) of the Xilinx Zynq UltraScale+ XCZU4EV MPSoC, leveraging the tight integration between the hardware logic and the ARM Cortex-A53 Processing System (PS). The PS serves as the system controller and orchestrator, managing configuration, DMA programming, input data movement, and post-processing. Communication between the processor and the accelerator is facilitated via the General Purpose AXI Master interface (M_AXI_HPM) using memory-mapped AXI4-Lite Control and Status Registers (CSRs). Bulk data transfer—such as reading CSC-formatted columns or writing updated neuron states—is performed via an AXI4 High-Performance (HP) slave interface connected to shared DDR4 memory. [1]

1.2. System Context and Execution Flow

The CGRA is designed as a tightly integrated hardware accelerator under full control of the ARM Cortex-A53 subsystem. The typical execution flow consists of:

1. **Configuration Stage**
   The HPS writes bitstream (context words) into the CGRA configuration memory via the CSR interface. These bitstream encode the spatial and temporal behavior of the 4×4 PE mesh.
2. **Data Preparation**
   SNN weight matrices are stored in **Compressed Sparse Column (CSC)** format in DDR memory. Input spike vectors and neuron states are prepared and aligned for DMA-based transfer.
3. **Execution Phase**
   The CGRA performs:
   - Sparse matvec operations using CSC:

$$y = W \times x \tag{1.1}$$

   where x is the binary spike vector.

   - LIF neuron updates in each PE:

$$v[t+1] = \beta \cdot v[t] + I[t] - \lambda \tag{1.2}$$

   including threshold and reset behavior.

4. **Result Collection**
   Updated membrane potentials and spike outputs are written back to DDR for use by the next simulation timestep or software post-processing.

1.3. Target Software Environment

The accelerator is co-designed with a two-phase software stack:

- **Phase 1: Bare-metal C on ARM Cortex-A53**

  Allows direct CSR access, minimal latency, and fast prototyping of SNN kernels.

- **Phase 2: Embedded Linux on ARM**

  Reuses the exact same hardware interface and register map.

  A Linux driver or user-space memory-mapped I/O library provides access to:

  - Configuration registers
  - DMA command queues
  - Performance counters
  - Bitstream-loading APIs

1.4. Target Workload and Problem Scope

The CGRA accelerator is optimized for **sparse SNN layer updates**, specifically weight matrices:

$$W \in \mathbb{R}^{128x128}$$

stored in **Compressed Sparse Column (CSC) format**. This representation allows the accelerator to process only active presynaptic spikes by scanning the corresponding nonzero column entries, thus avoiding dense computation.

The following operations constitute the primary workload:

- Sparse synaptic accumulation using the CSC tuple

$$(col_ptr, row_idx, val)$$

- LIF neuron updates for all neurons in the layer
- Binary spike generation for subsequent layers or timesteps

The 4×4 PEs mesh supports scalable tiling strategies for matrices larger than the mesh dimension, enabling iterative processing of neuron blocks.

1.5. Project Objectives

The primary objective of this project is to design and implement a CGRA-based accelerator IP featuring an APB control interface and an AXI4 high-bandwidth DMA engine for data movement. The major design goals include:

- **Development of a 4×4 Processing Element (PE) mesh** incorporating wide-precision (40-bit) saturating arithmetic to support high-dynamic-range SNN computations.
- **Integration of a dedicated hardware Leaky Integrate-and-Fire (LIF) unit** within each PE to accelerate neuron update equations, including membrane potential evolution v[t+1] and spike generation.
- **Implementation and verification of a "hot-reloadable" configuration mechanism** using double-buffered context memory to allow seamless bitstream switching without interrupting system operation.
- **Validation of the complete accelerator on the DE10-Standard FPGA platform**, ensuring correct functional behavior, AXI protocol compliance, and performance advantages over CPU-based execution.

2. LITERATURE REVIEW AND BACKGROUND

2.1. Overview of Coarse-Grained Reconfigurable Architectures (CGRAs)

Coarse-Grained Reconfigurable Architectures (CGRAs) represent a class of programmable hardware structures positioned between traditional fine-grained FPGAs and fixed-function ASICs. Unlike LUT-based FPGA fabrics, CGRAs are composed of **word-level functional units**, called Processing Elements (PEs), organized in a regular array. Each PE typically includes arithmetic logic capabilities, local register files, and configurable routing paths that facilitate parallel computation.

Modern machine learning and signal processing algorithms frequently demand high throughput and reconfigurable computation patterns. Fine-grained FPGAs can satisfy these requirements but suffer from high configuration overhead, routing congestion, and inefficient mapping of arithmetic-heavy workloads. CGRAs mitigate these issues by operating at the arithmetic-operator granularity, eliminating much of the redundant configuration and interconnect complexity found in LUT-based architectures. This provides improved energy efficiency, deterministic performance, and reduced implementation effort for many computational kernels.

A typical CGRA executes computations in three primary phases:

1. **Configuration Mapping** – Where context words define the operation of each PE.
2. **Spatial/Temporal Scheduling** – Where operations are mapped to specific cycles and PEs.
3. **Data Routing and Execution** – Where inputs flow through the mesh according to the configured routing scheme.

This organization allows the CGRA to support complex, multi-stage computations without the excessive reconfiguration that limits traditional FPGAs. The fundamental advantage lies in its ability to execute nested loops, dataflow

workloads, and sparse computations efficiently using deterministic pipelines and repeated context reuse.

2.2. Processing Element (PE) and Mesh Network Architecture

2.2.1. Overview of the CGRA Processing Element

The PE is the fundamental building block of the CGRA fabric. In the proposed design, each PE integrates:

- A **40-bit saturating multiply-accumulate (MAC) unit** for synaptic weight accumulation.
- A **32-bit ALU** for auxiliary arithmetic and logical operations.
- A dedicated **Leaky Integrate-and-Fire (LIF) neuron unit** for biological neural computation.
- A small **scratchpad memory** to store intermediate values and neuron states.
- A **register file** for low-latency operand access.
- A **context register** that receives cycle-level control words during execution.

This structure is analogous to the "partial product + adder tree + pipeline stage" decomposition seen in your multiplier theory chapter, but adapted here to represent **compute + routing + neuron-update** stages of the CGRA. [1] [2]

2.2.2. Mesh Routing Network and Data Movement

Much like the Wallace Tree multiplier organizes partial products into parallel reduction stages, the CGRA organizes dataflow through a **mesh router network** that supports multi-hop propagation of operands and spikes [1]. Each router includes:

- Five ports (North, East, South, West, Local),
- Input FIFOs per direction,
- Support for unicast and **multicast** delivery,

- Backpressure signaling,
- Deadlock-free routing policies.

This network enables efficient distribution of spikes and partial sums among PEs. For example, when a presynaptic neuron fires, the associated spike event may need to reach multiple downstream neurons; multicast routing eliminates redundant memory accesses and improves overall throughput.

A conceptual mesh routing stage is illustrated in **Figure 1**, equivalent in function to the “partial product organization" diagram in your multiplier theory but for dataflow routing.

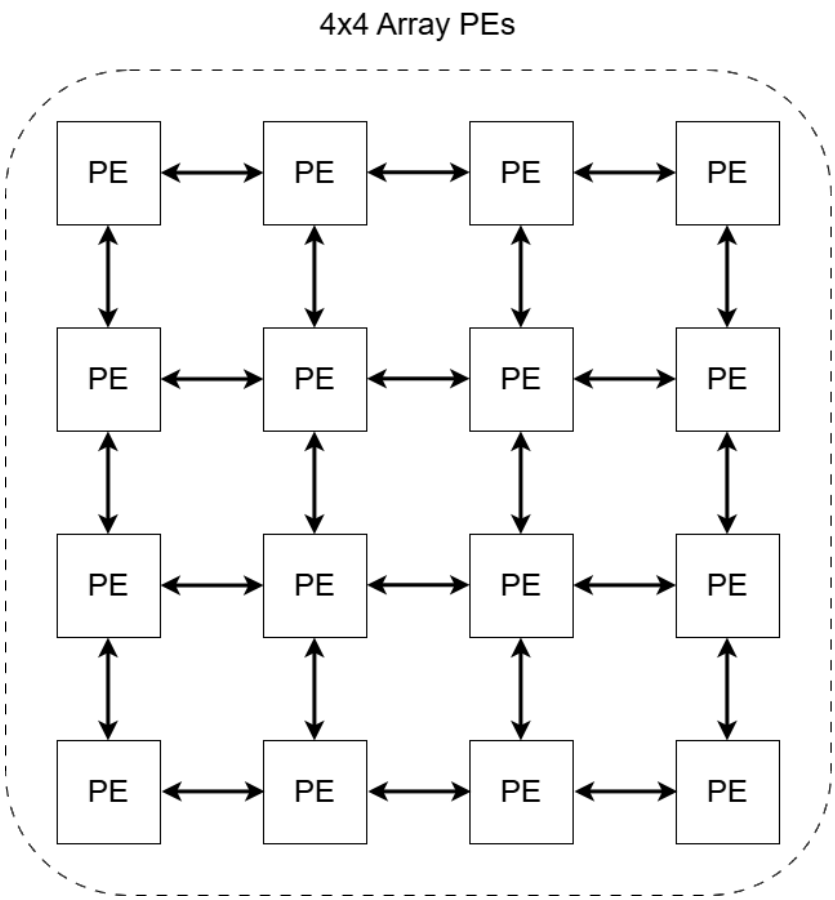

Figure 1: Mesh routing for 4x4 CGRA

2.3. Spiking Neural Networks (SNNs)

Spiking Neural Networks (SNNs) represent the third generation of neural network models, in which information is encoded and processed using discrete spike events rather than continuous-valued activations. Unlike traditional Artificial Neural Networks (ANNs) that operate on static real-valued vectors, SNNs introduce an explicit notion of time, and neuronal state evolves across discrete timesteps according to dynamical equations.

This temporal and event-driven formulation allows SNNs to more closely resemble biological neural systems, where neurons communicate through short electrical impulses (spikes), and computation is often sparse in both time and space. From a hardware perspective, these characteristics are attractive because they enable data-dependent, sparse computation, which can reduce switching activity and energy consumption if mapped onto an appropriately designed architecture. [5] [6]

A conceptual overview of a single SNN neuron receiving spike inputs from multiple presynaptic neurons, accumulating them into a membrane potential, and emitting an output spike when a threshold is crossed can be illustrated in *Figure 2*

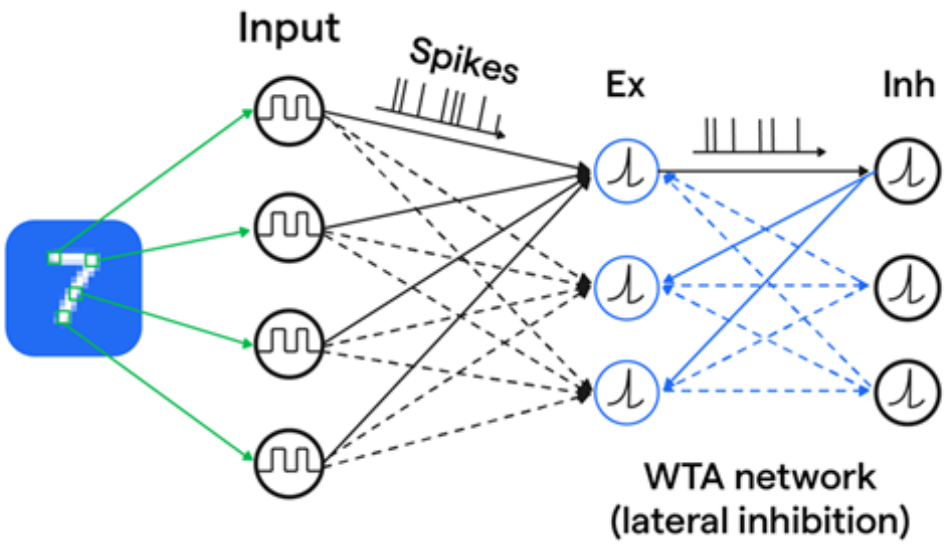

Figure 2: Spiking Neural Networks (SNNs)

2.3.1. Neuron State, Membrane Potential, and Spike Events

In SNNs, each neuron maintains a **membrane potential** v(t), which is a state variable summarizing the neuron's internal condition at time t. The membrane potential integrates incoming synaptic currents and decays (or "leaks") over time. When v(t) exceeds a firing threshold V_{th}, the neuron emits a spike and its membrane potential is typically reset. [4]

At a high level, the behavior of a neuron can be separated into three phases per timestep:

1. **Integration:**
   Incoming spikes from presynaptic neurons cause weighted increments to the membrane potential.
2. **Leak:**
   The membrane potential decays over time to model passive leakage of charge.
3. **Threshold and Reset:**
   If the membrane potential exceeds a threshold, a spike is generated and the potential is reset (often to a baseline value).

In discrete-time SNN models used for digital hardware, these phenomena are approximated with simple difference equations, making them suitable for fixed-point arithmetic and parallel implementation. [6]

2.3.2. Leaky Integrate-and-Fire (LIF) Neuron Model

Among many neuron models proposed in computational neuroscience, the Leaky Integrate-and-Fire (LIF) model strikes a balance between biological plausibility and implementation simplicity. It captures the essential features of integration, leakage, and threshold-based spiking without the complexity of biophysical models such as Hodgkin–Huxley.

In discrete time, the LIF neuron is commonly modeled as:

$$v[t+1] = \beta \cdot v[t] + I[t] - \lambda \tag{2.1}$$

where:

- *v[t]* is the membrane potential at discrete timestep t,
- $\beta \in (0,1)$ is the **leak factor**, controlling how quickly the potential decays over time,
- *I[t]* is the **synaptic input current** at time t (sum of weighted spikes),
- λ is a constant term modeling tonic leak or bias.

A spike is generated at time t+1 if:

$$v[t+1] \geq V_{th} \tag{2.2}$$

where V_{th} is the **firing threshold**. After a spike, the membrane potential is typically reset. [4]

2.3.3. Information Encoding Schemes

Unlike Artificial Neural Networks (ANNs) where information is transmitted as continuous floating-point values (activations), SNNs communicate via binary spike events. Therefore, a method is required to encode real-world data (such as pixel intensities in an image) into a train of spikes. There are two primary encoding paradigms:

- Rate Coding (Frequency Coding):

Information is conveyed by the firing rate of the neurons. A higher input value corresponds to a higher frequency of spikes over a given time window. While robust and easy to implement, rate coding can be inefficient as it requires a longer simulation time to estimate the precise value of a signal. [5]

- Temporal Coding (Time-to-First-Spike):

Information is encoded in the precise timing of individual spikes relative to a stimulus onset. For example, a stronger stimulus might cause a neuron to fire earlier than a weaker stimulus. This method is highly sparse and energy-efficient, as it relies on fewer spikes to transmit the same amount of information, making it particularly attractive for hardware accelerators aiming for ultra-low latency. [6]

In the context of this CGRA accelerator, the architecture is designed to be encoding-agnostic, supporting sparse spike vectors regardless of whether they were generated via rate or temporal schemes, provided they are formatted into the appropriate Compressed Sparse Column (CSC) representation.

2.3.4. Training Methodologies for SNNs

Since this project focuses on a hardware accelerator for *inference*, it is essential to understand how the parameters (weights and thresholds) are obtained. There are two main approaches to training the SNNs that this accelerator will execute:

1. ANN-to-SNN Conversion:

   A conventional ANN is trained using standard backpropagation on a GPU. Once trained, the floating-point weights are mapped to SNN parameters. The activation of an ANN neuron is approximated by the firing rate of an SNN neuron. This method achieves high accuracy on complex tasks (like ImageNet) but may require higher latency during inference to approximate the values accurately. [8] [9]

2. Direct Training (Spike-Based Learning):

   The SNN is trained directly in the spiking domain. This can be done using:

   - **Surrogate Gradients:** Approximating the non-differentiable spike function to allow backpropagation through time (BPTT).
   - **Unsupervised Learning (STDP):** Spike-Timing-Dependent Plasticity is a local learning rule where the weight of a synapse is adjusted based on the relative timing of the presynaptic and postsynaptic spikes. If the presynaptic neuron fires just before the postsynaptic neuron, the connection is strengthened (Long-Term Potentiation).

The proposed CGRA accelerator is capable of executing SNN models trained via either method, as the resulting weights are loaded into the system memory prior to execution. [7] [10]

2.3.5. Computational Advantages over Von Neumann Architectures

The theoretical motivation for designing a specific CGRA for SNNs lies in the fundamental differences between neuromorphic and Von Neumann computations:

- **Event-Driven Processing:** In standard matrix multiplication (used in ANNs), every weight is multiplied by every activation, even if the activation is zero (or near zero). In contrast, SNNs are event-driven; computation in the LIF unit is only triggered when an input spike arrives. If no spike occurs, the PE remains idle, saving dynamic power.
- **Sparsity Exploitation:** Biological neural activity is incredibly sparse (often <1% activity). By using CSC format and a multicast routing network, this architecture avoids fetching "zero" values, overcoming the "Memory Wall" bottleneck that limits the performance of standard CPUs and GPUs on sparse workloads. [11]
- **Locality of Reference:** The LIF neuron model requires storing the membrane potential v[t] locally. In a CPU/GPU, this state must be constantly shuffled back and forth to main memory. In this CGRA, the state is kept in the local PE scratchpad (SPM), drastically reducing data movement energy. [6]

2.3.6. Energy Efficiency and Event-Driven Processing

The fundamental advantage of SNNs lies in their event-driven nature, which contrasts sharply with the dense, frame-based processing of traditional ANNs. In ANNs, every neuron in a layer computes an activation value every cycle, regardless of whether the information is relevant. SNNs, however, are quiescent by default; they consume dynamic power only when a spike event occurs.

This characteristic aligns naturally with the "sparsity" observed in biological systems and modern neuromorphic sensors (like DVS or SPADs), where data is sparse in both time and space. By processing information only upon the arrival of relevant input spikes, SNNs can achieve significant reductions in switching activity and energy consumption. For hardware accelerators like the proposed CGRA, this allows for fine-grained clock gating or logic disabling during idle periods between spikes, overcoming the "memory wall" and power constraints typical of edge computing environments. [5] [6]

2.3.7. Challenges in SNN Training and Deployment

Despite their efficiency, deploying SNNs presents unique challenges compared to standard deep learning:

- **Training Complexity:** Unlike ANNs which use standard backpropagation, SNNs require specialized training methods due to the non-differentiable nature of discrete spikes. Common strategies include *ANN-to-SNN Conversion* (converting a pre-trained ANN to SNN, often at the cost of higher latency) and *Direct Training* using surrogate gradients (approximating the spike function to enable backpropagation through time). [5]
- **Hardware Mismatch:** Standard hardware is rarely optimized for sparse, irregular memory access patterns inherent in SNNs. This "algorithm-hardware mismatch" necessitates custom architectures - like the CGRA proposed in this project - that support efficient multicast routing and local state management to fully exploit the theoretical efficiency gains of SNNs. [7]

3. SYSTEM ARCHITECTURE

3.1. SoC System Integration Block Diagram

The system integrates the CGRA Accelerator IP with the host processor (ARM Cortex) via the following interfaces:

- **AXI4 Interconnect:** Facilitates high-bandwidth communication between the CPU and the external **DRAM** for data storage.
- **AXI-to-APB Bridge:** Translates transactions from the high-speed system bus (AXI) to the peripheral bus (APB). [13]
- **APB Interface:** Serves as the control interface for configuring the **CGRA Accelerator IP** and accessing its Control/Status Registers (CSRs).
- **DRAM:** Acts as the shared main memory for holding large neural network weights and input data.

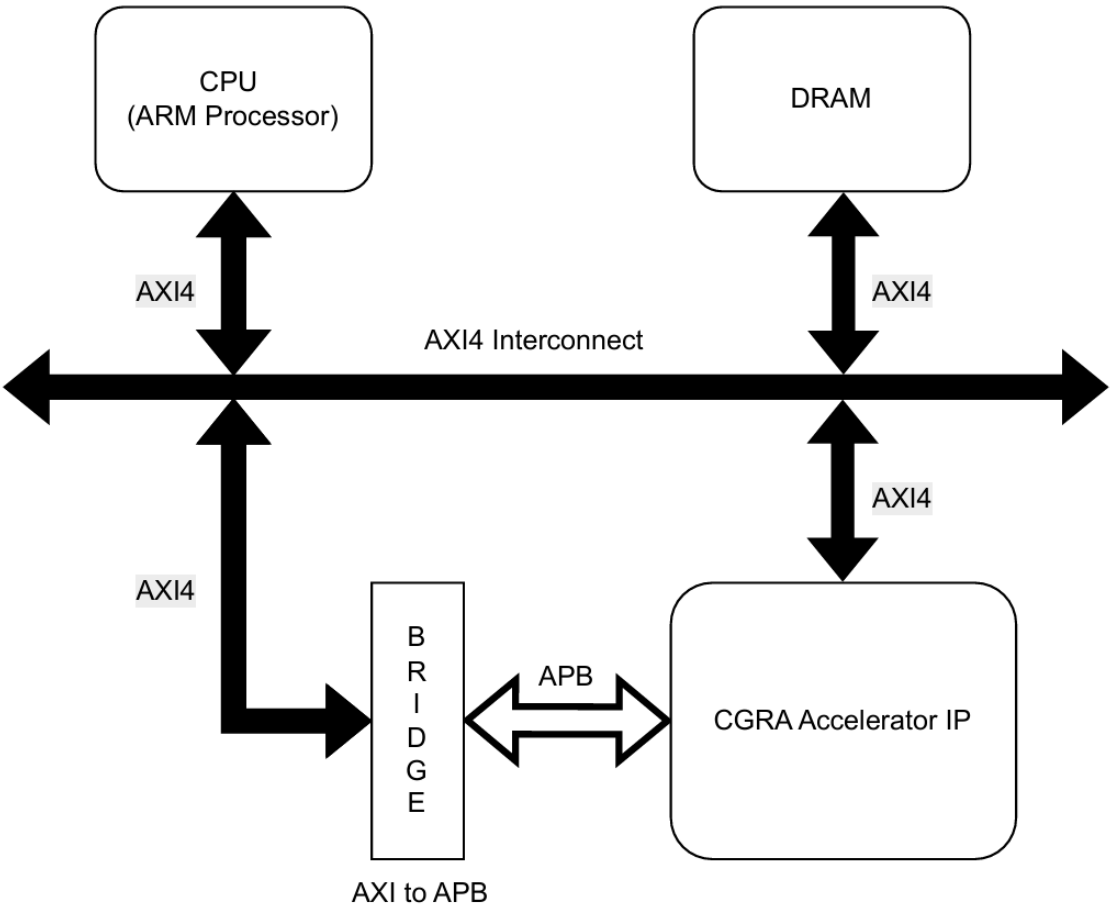

Figure 3: SoC System Integration Block Diagram

3.2. Block Diagram Detail of CGRA Accelerator IP

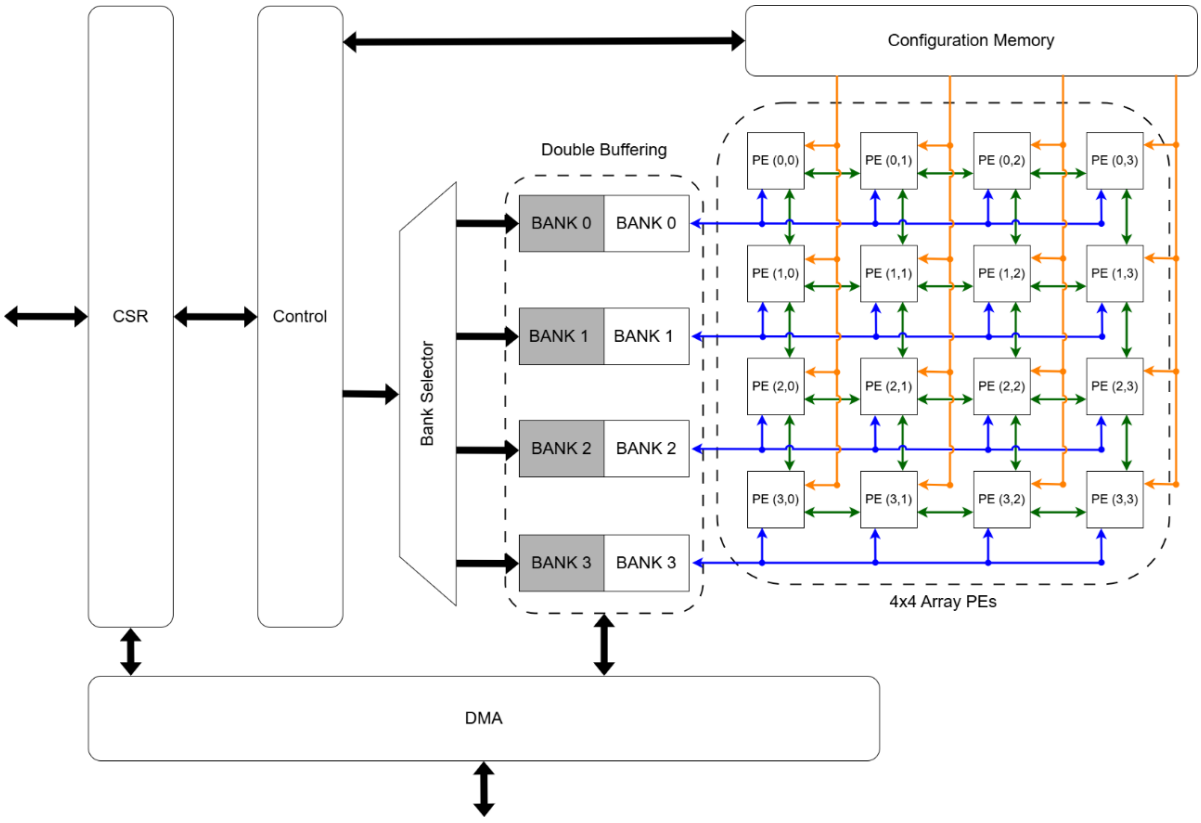

Figure 4: Block Diagram for CGRA Accelerator IP

The internal architecture of the CGRA is partitioned into four major subsystems, designed to maximize data throughput and minimize stall cycles:

1. **Global Control and CSR:** This module acts as the interface to the host processor. It manages the accelerator's lifecycle, handling interrupt generation, status monitoring, and the decoding of commands via the APB slave interface.
2. **Double-Buffered Configuration Memory:** To support rapid reconfiguration required by multi-layer neural networks, the configuration memory employs a double-buffering scheme. This allows the DMA to pre-fetch the bitstream for the *next* layer into the "back" buffer while the *current* layer is executing from the "front" buffer, effectively hiding reconfiguration latency.
3. **DMA Engine and Bank Selector:** The Direct Memory Access (DMA) unit operates as an AXI4 Master to fetch data from external DRAM. The **Bank Selector** logic routes these incoming data streams to the appropriate on-chip memory banks based on the target row index, ensuring efficient data scattering.

4. **Row-Banked Tile Memory:** The global tile memory is physically partitioned into four independent SRAM banks (Bank 0 to Bank 3). Each bank is dedicated to a specific row of the PE array (e.g., Bank 0 serves Row 0). This **Row-Banked** organization eliminates memory contention, enabling all four rows of PEs to fetch operands simultaneously in a single clock cycle.
5. **4×4 PE Array:** The computational core consists of 16 Processing Elements arranged in a 2D mesh topology. Each PE (indexed 0,0 to 3,3) is tightly coupled with the routing logic to perform parallel neuromorphic computations.

3.3. Processing Element (PE) Tile Architecture

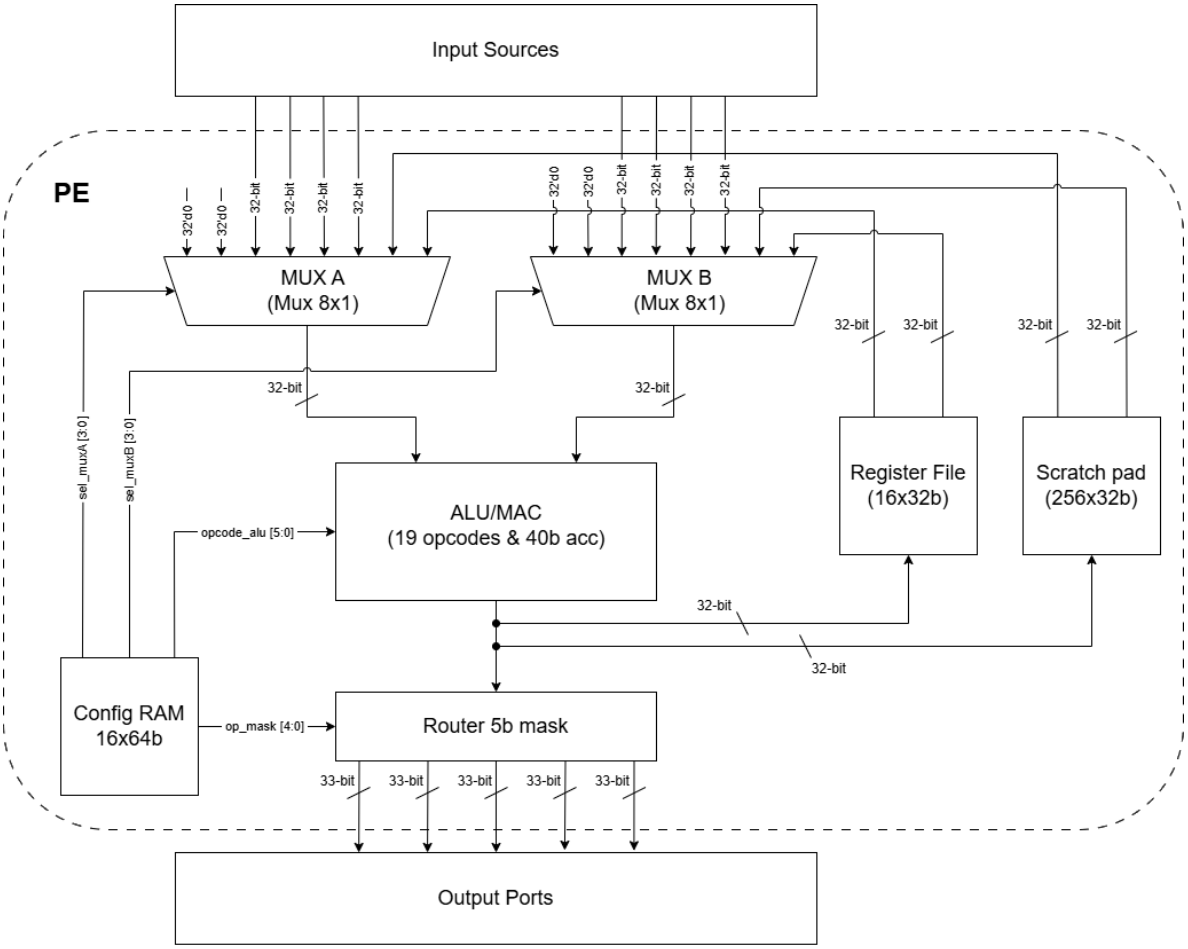

Figure 5: Block diagram for each PE

Each PE tile forms the fundamental compute node in the CGRA.

A tile integrates a compute datapath, local scratchpad memory, an optional LIF neuron unit, and a fully featured 5-port NoC router to exchange data with neighboring tiles.

This enables highly parallel, spatial execution of kernels such as convolutions, dot-products, and spike-based operations.

The PE datapath consists of:

- **Input:** Data arrives from 4 neighbors (N/E/S/W) with valid signals
- **Decode:** Configuration frame sets operation and routing
- **Operand Select:** Two operands selected via mux
- **Compute:** ALU performs operation, updates accumulator if needed
- **Predicate:** Conditionally gates write-back based on comparison flags
- **Write-back:** Result stored to RF or SPM
- **Output:** Result packed with routing header, sent to neighbors via route mask

3.4. Instruction Set Architecture (ISA)

The CGRA operates on a 64-bit Very Long Instruction Word (VLIW) architecture. Each Processing Element (PE) executes one instruction per clock cycle, defined by a 64-bit configuration frame loaded into its local context memory. The ISA supports 19 distinct operations ranging from standard arithmetic to specialized neuromorphic functions.

3.4.1. Configuration Frame Format

The 64-bit configuration word controls the PE's datapath, including operand selection, ALU operation, destination routing, and immediate value generation. The bit mapping is defined as follows:

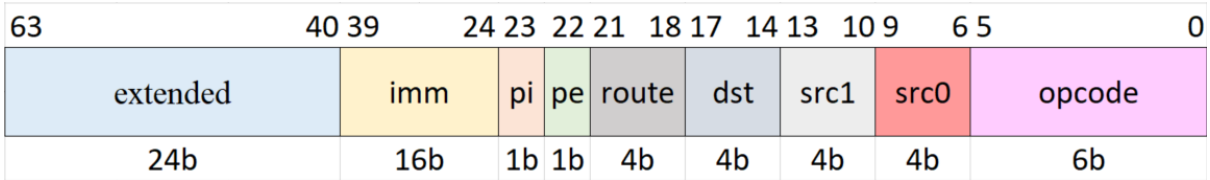

Figure 6: Configuration Frame

Field	Bits	Width	Description
opcode	[5:0]	6	ALU Operation Code
src0_sel	[9:6]	4	Source Select for Operand A (RF, Neighbors, SPM, Immediate)

src1_sel	[13:10]	4	Source Select for Operand B
dst_sel	[17:14]	4	Write-back address for Local Register File (RF)
route_mask	[21:18]	4	Multicast Output Mask (Bit 0: North, 1: East, 2: South, 3: West)
pred_en	[22]	1	Enable Predicated Execution
pred_inv	[23]	1	Invert Predicate Logic
imm	[39:24]	16	Signed 16-bit Immediate Value
extended	[63:40]	24	Reserved for future extensions or extra routing bits

Table 1: Configuration Summary

3.4.2. Opcode Summary

The ALU supports 19 operations categorized into arithmetic, logic, flow control, and neuromorphic updates. All operations execute in a single cycle.

Opcode	**Mnemonic**	**Operation / Description**
0	NOP	No Operation
1	ADD	A + B (Saturating) - Adds two operands with overflow protection
2	SUB	A - B (Saturating) - Subtracts operand B from A
3	MUL	A × B - Performs 32-bit multiplication
4	MAC	Acc += A × B - Multiply-Accumulate into 40-bit internal accumulator
5	AND	A & B - Bitwise AND
6	OR	`A
7	XOR	A ^ B - Bitwise XOR
8	SHL	A << B[4:0] - Logical Shift Left
9	SHR	A >>> B[4:0] - Arithmetic Shift Right (preserves sign)
10	CMP_GT	(A > B) ? 1 : 0 - Greater Than comparison

11	CMP_LT	(A < B) ? 1 : 0 - Less Than comparison
12	CMP_EQ	(A == B) ? 1 : 0 - Equality comparison
13	LOAD_SPM	RF ← SPM[A] - Load data from local Scratchpad Memory to Register File
14	STORE_SPM	SPM[A] ← B - Store data from operand B into Scratchpad Memory at address A
15	ACC_CLR	Acc = 0 - Clears the internal 40-bit accumulator
16	PASS0	Output = A - Passes operand A to the output (for routing)
17	PASS1	Output = B - Passes operand B to the output (for routing)
18	LIF	Spike = LIF(v, i) - Executes Leaky Integrate-and-Fire neuron update and threshold check

Table 2: Opcodes of ALU

4. HARDWARE DESIGN

4.1. Top Module (cgra_top.sv)

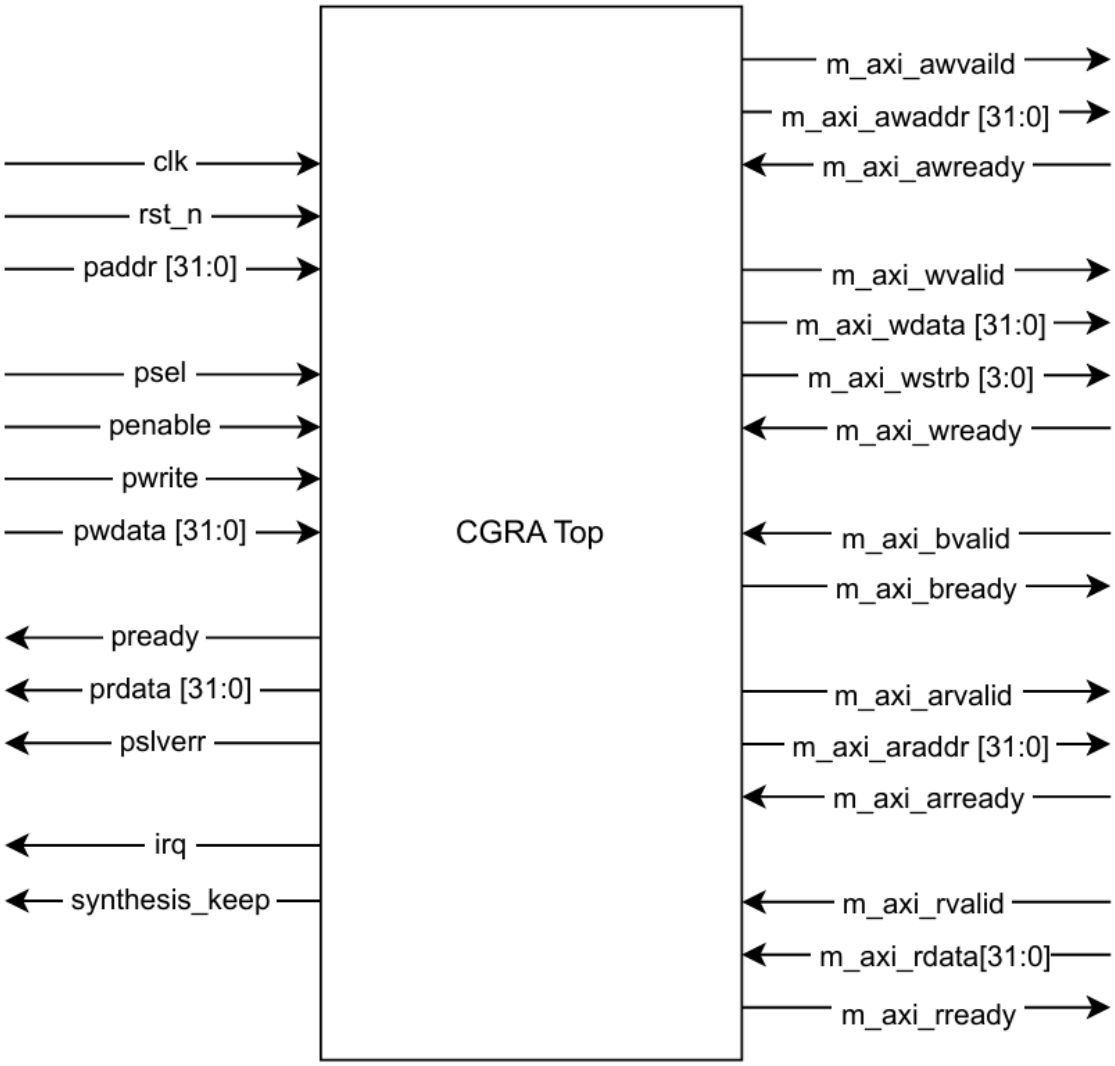

Figure 7: Top Module CGRA

Signal Name	Direction	Width	Description
clk	Input	1	System clock
rst_n	Input	1	Active-low synchronous reset
APB Slave Interface			
psel	Input	1	APB select signal
penable	Input	1	APB enable signal
pwrite	Input	1	APB write enable

paddr	Input	ADDR_WIDTH	APB address bus
pwdata	Input	32	APB write data
prdata	Output	32	APB read data
pready	Output	1	APB ready signal
pslverr	Output	1	APB slave error
AXI4-Lite Master (DMA Interface to External RAM)			
m_axi_awaddr	Output	ADDR_WIDTH	Write address
m_axi_awvalid	Output	1	Write address valid
m_axi_awready	Input	1	Write address ready
m_axi_wdata	Output	32	Write data
m_axi_wstrb	Output	4	Write strobes
m_axi_wvalid	Output	1	Write data valid
m_axi_wready	Input	1	Write data ready
m_axi_bvalid	Input	1	Write response valid
m_axi_bready	Output	1	Write response ready
m_axi_araddr	Output	ADDR_WIDTH	Read address
m_axi_arvalid	Output	1	Read address valid
m_axi_arready	Input	1	Read address ready
m_axi_rdata	Input	32	Read data
m_axi_rvalid	Input	1	Read data valid
m_axi_rready	Output	1	Read data ready
System Signals			
irq	Output	1	Interrupt output (DMA or Control Unit done)

synthesis_keep	Output	1	Prevent optimization of unconnected outputs

Table 3: Signals of top module CGRA

- **Description:**

  The cgra_top module serves as the hardware wrapper for the entire accelerator, exposing standard AMBA interfaces to the system interconnect. It instantiates the core 4x4 array, the global control unit, the DMA engine, and the memory subsystems. It manages the clock domain crossing (if applicable) and reset synchronizers. The module creates two primary interfaces: an AXI4-Lite Slave for memory-mapped control/status register (CSR) access by the CPU, and an AXI4 Master for high-bandwidth bulk data transfers to and from the external DDR memory.

4.2. CSR (cgra_apb_csr.sv)

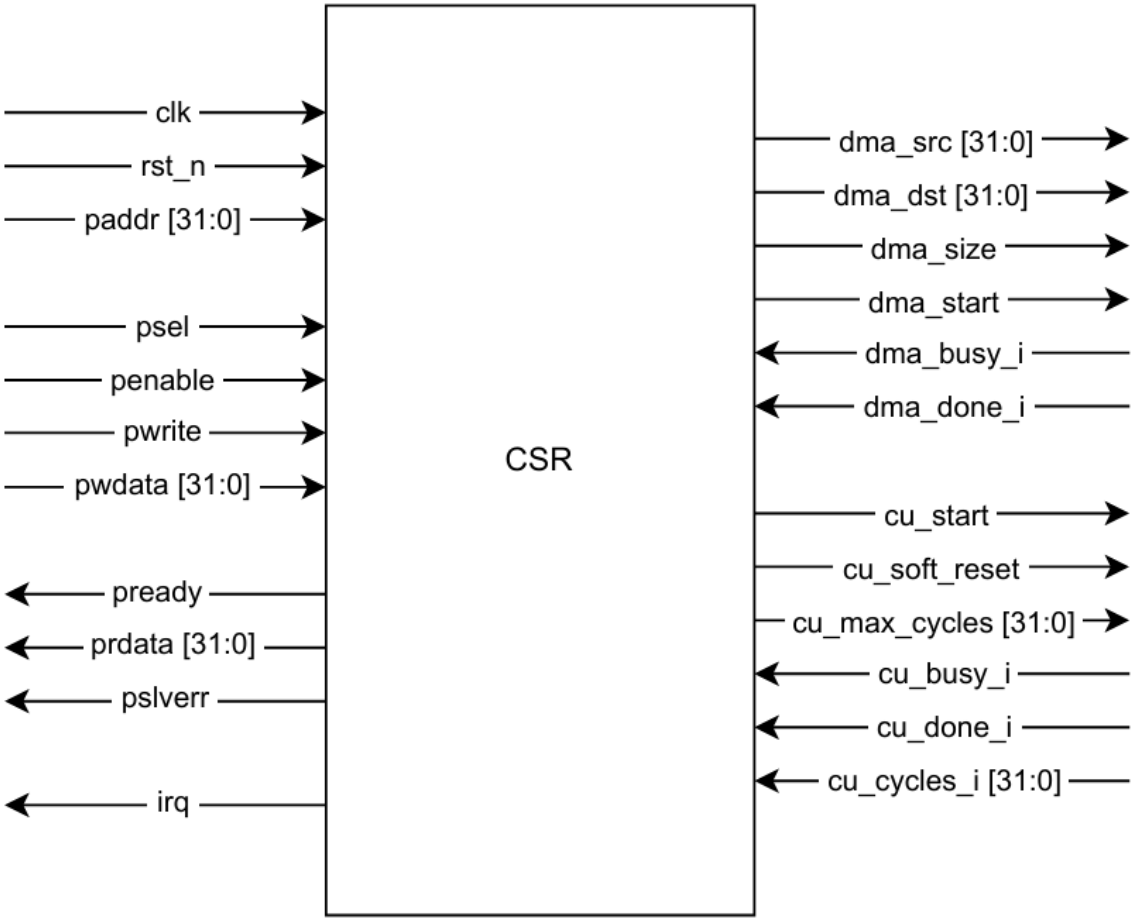

Figure 8: Module CSR (APB Slave)

Signal Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Active-low reset
APB Interface			
psel	Input	1	APB Select
penable	Input	1	APB Enable
pwrite	Input	1	APB Write
paddr	Input	ADDR_WIDTH	APB Address
pwdata	Input	DATA_WIDTH	APB Write Data
prdata	Output	DATA_WIDTH	APB Read Data
pready	Output	1	APB Ready
pslverr	Output	1	APB Slave Error
DMA Config			

dma_src	Output	32	DMA source address
dma_dst	Output	32	DMA destination address
dma_size	Output	32	Transfer size in bytes
dma_start	Output	1	DMA start pulse (auto-clearing)
dma_busy_i	Input	1	DMA busy status
dma_done_i	Input	1	DMA done status
Control Unit Config			
cu_start	Output	1	Execution start pulse
cu_soft_reset	Output	1	Soft reset signal
cu_max_cycles	Output	32	Programmable timeout limit
cu_busy_i	Input	1	Control Unit busy status
cu_done_i	Input	1	Control Unit done status
cu_cycles_i	Input	32	Cycle counter value
Interrupt			
irq	Output	1	Combined interrupt request

Table 4: Signals of CSR Module

- **Description:**

The Control and Status Register (CSR) block implements the register map accessible by the host processor. It translates AXI4-Lite (or APB) read/write transactions into internal control signals. Key registers include the Control Register (to start/stop execution), Status Register (monitoring busy/done states and error flags), and Configuration Registers (setting the base address and size for bitstreams). It also manages interrupt generation, asserting the irq line when a workload completes or an error occurs.

4.3. Control Unit (cgra_control_unit.sv)

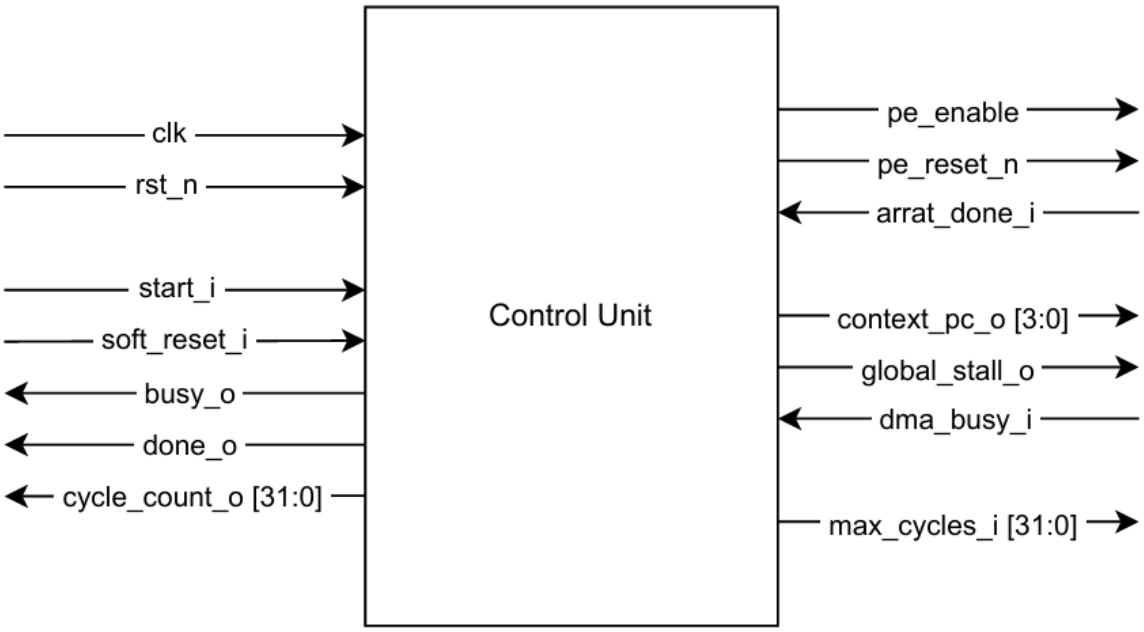

Figure 9: Module Control Unit

Signal Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Reset
CSR Interface			
start_i	Input	1	Start pulse
soft_reset_i	Input	1	Soft reset signal
busy_o	Output	1	Busy status
done_o	Output	1	Done status
cycle_count_o	Output	32	Cycle count
Array Interface			
pe_enable	Output	1	Global PE enable
pe_reset_n	Output	1	PE array reset
array_done_i	Input	1	Done signal from array
Flow Control			
context_pc_o	Output	PC_WIDTH	Current Context PC
global_stall_o	Output	1	Global stall signal
dma_busy_i	Input	1	DMA busy status

Config			
max_cycles_i	Input	32	Timeout cycle limit

Table 5: Signals of Control Unit Module

- **Description:**

This module is the central sequencer of the accelerator. It implements a Finite State Machine (FSM) that orchestrates the execution flow:

- **Configuration Phase:** Loading the context words from the configuration memory into the PEs.
- **Execution Phase:** Advancing the global context_pc counter to cycle through the scheduled instructions.
- **Synchronization:** Managing the double-buffered context switching to ensure zero-stall transitions between time slots. It generates the global stall signals and enforces the programmable timeout limit (max_cycles_i) to prevent deadlocks during experimental kernels.

4.4. DMA Engine (cgra_dma_engine.sv)

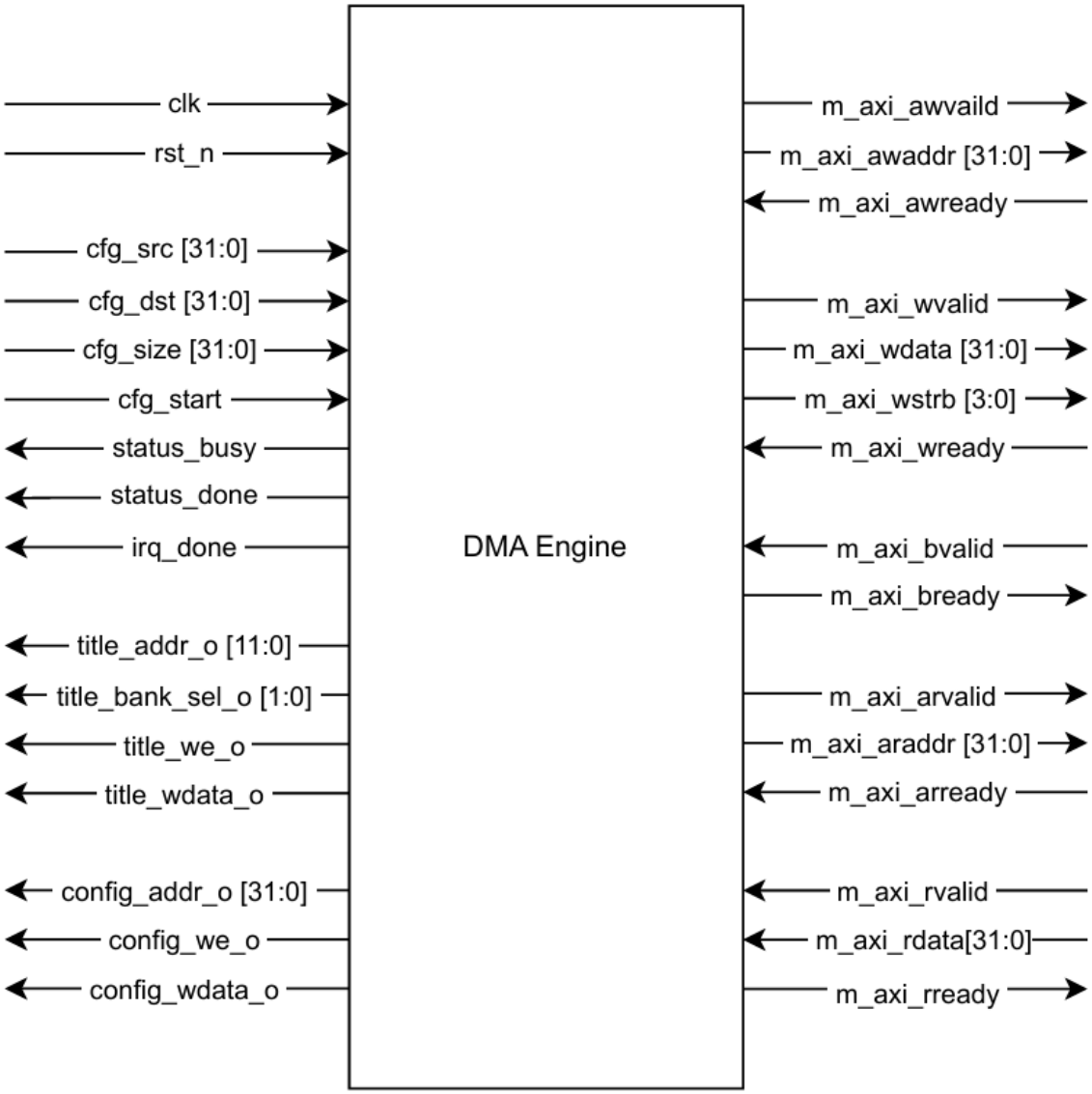

Figure 10: Module DMA Engine

Signal Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Active-low reset
Config			
cfg_src	Input	32	Source address
cfg_dst	Input	32	Destination address
cfg_size	Input	32	Transfer size
cfg_start	Input	1	Start signal
status_busy	Output	1	Busy status

status_done	Output	1	Done status
irq_done	Output	1	Interrupt pulse on completion
AXI4-Lite Master *(Standard AXI Master Signals)*			
m_axi_awaddr	Output	ADDR_WIDTH	Write address
m_axi_awvalid	Output	1	Write address valid
m_axi_awready	Input	1	Write address ready
m_axi_wdata	Output	32	Write data
m_axi_wstrb	Output	4	Write strobes
m_axi_wvalid	Output	1	Write data valid
m_axi_wready	Input	1	Write data ready
m_axi_bvalid	Input	1	Write response valid
m_axi_bready	Output	1	Write response ready
m_axi_araddr	Output	ADDR_WIDTH	Read address
m_axi_arvalid	Output	1	Read address valid
m_axi_arready	Input	1	Read address ready
m_axi_rdata	Input	32	Read data
m_axi_rvalid	Input	1	Read data valid
m_axi_rready	Output	1	Read data ready
Local Memory IF (Interface to Tile Memory)			
tile_addr_o	Output	12	Tile memory address
tile_bank_sel_o	Output	2	Bank select (0-3)
tile_we_o	Output	1	Write enable
tile_wdata_o	Output	32	Write data
Config IF (Interface to PE Array)			
config_addr_o	Output	32	Configuration address
config_we_o	Output	1	Configuration write enable
config_wdata_o	Output	32	Configuration write data

Table 6: Signals of DMA Engine

- **Description:**

The DMA engine is a high-performance data mover designed to alleviate the "memory wall." It operates as an AXI4 Master with a producer-consumer FIFO

architecture. It supports 1D and 2D transfer patterns, allowing it to efficiently fetch Compressed Sparse Column (CSC) data structures (col_ptr, row_idx, values) from external memory and scatter them into the on-chip tile memories. It features independent read/write channels to allow simultaneous loading of inputs and offloading of results.

4.5. Processing Element (cgra_pe.sv)

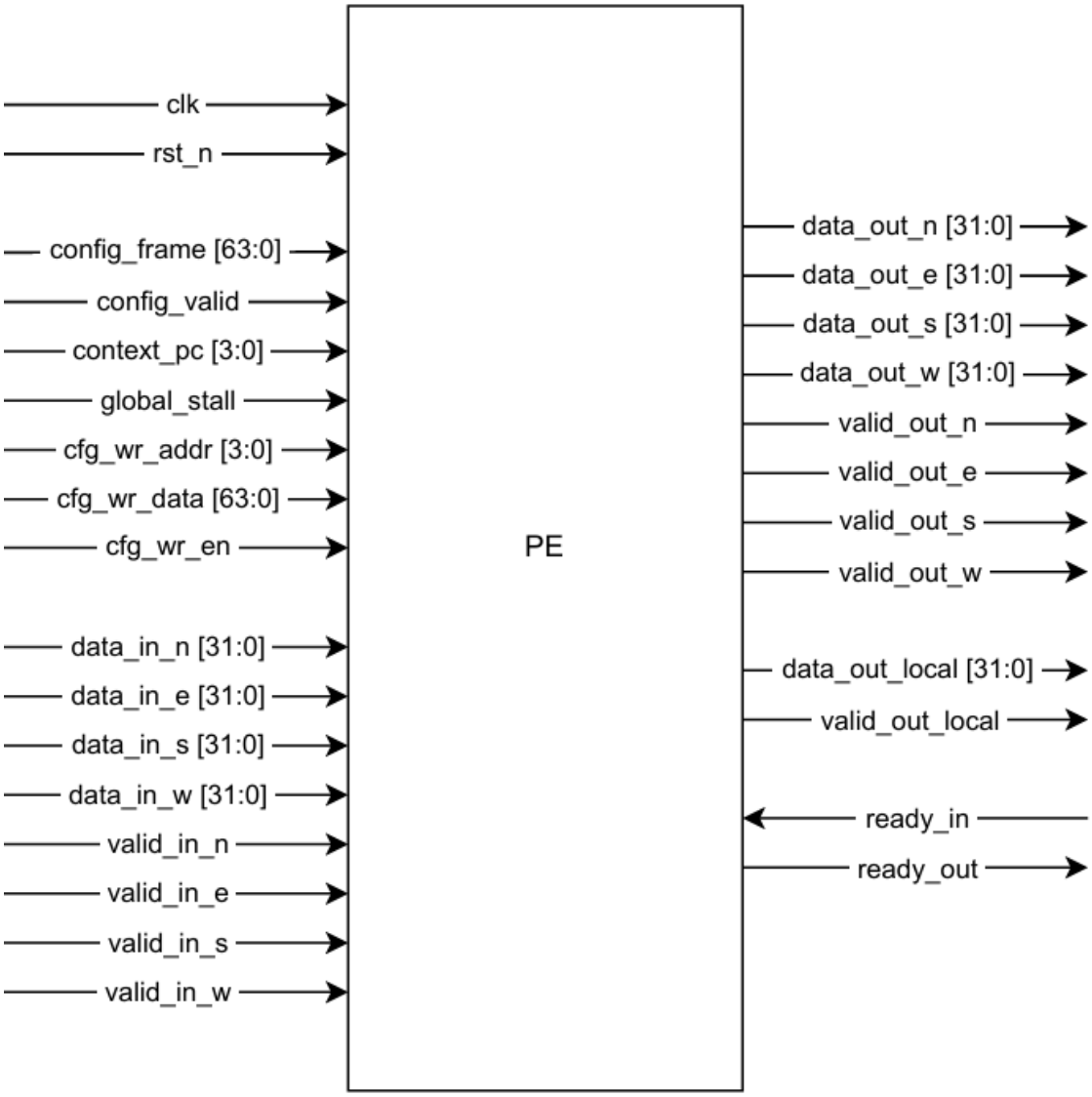

Figure 11: Module Processing Element

Signal Name	Direction	Width	Description
clk, rst_n	Input	1	Clock and Reset
config_frame	Input	64	Static configuration input
config_valid	Input	1	Configuration valid
context_pc	Input	PC_WIDTH	Context PC
global_stall	Input	1	Global Stall
Config Write			
cfg_wr_addr	Input	PC_WIDTH	Write Address
cfg_wr_data	Input	64	Write Data

cfg_wr_en	Input	1	Write Enable
Routing I/O			
data_in_n/e/s/w	Input	DATA_WIDTH	Data from neighbors
valid_in_n/e/s/w	Input	1	Valid from neighbors
data_out_n/e/s/w	Output	DATA_WIDTH	Data to neighbors
valid_out_n/e/s/w	Output	1	Valid to neighbors
Local I/O			
data_out_local	Output	DATA_WIDTH	Output to local router port
valid_out_local	Output	1	Valid to local router port
ready_in	Input	1	Ready from local router port
ready_out	Output	1	Ready to local router port

Table 7: Signals of Processing Element Module

- **Description:**

The PE is the computational heart of the CGRA. Each PE contains:

- **Neuromorphic ALU:** A hybrid unit capable of standard fixed-point arithmetic (ADD, SUB, MUL) and specialized LIF neuron updates (membrane potential integration, leak subtraction, and threshold check).
- **Scratchpad Memory:** A local register file for storing temporary variables and neuron states (v[t]), reducing the need for global memory access.
- Routing Logic: A crossbar switch that selects operands from local storage or one of the four neighbor ports (N, E, S, W).

The PE executes one instruction per cycle, defined by the 64-bit configuration word driven by the context memory.

4.6. Tile Memory (cgra_tile_memory.sv)

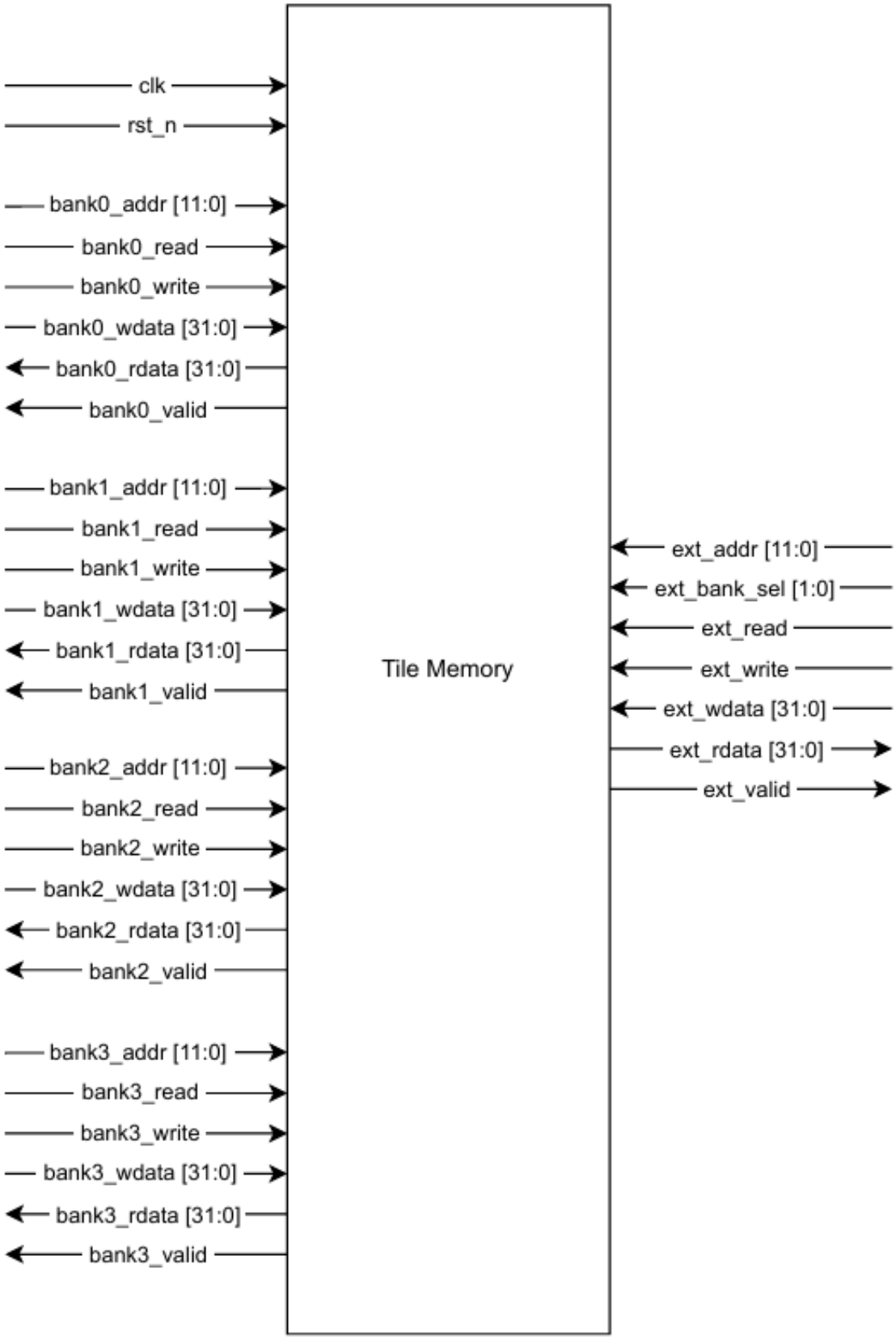

Figure 12: Module Tile Memory

Signal Name	Direction	Width	Description
clk	Input	1	Clock
rst_n	Input	1	Reset
Bank 0-3 Interfaces *(Repeated for Bank 0, 1, 2, 3)*			
bankX_addr	Input	ADDR_WIDTH	Read address (context_pc)
bankX_read	Input	1	Read enable
bankX_write	Input	1	Write enable
bankX_wdata	Input	DATA_WIDTH	Write data
bankX_rdata	Output	DATA_WIDTH	Read data
bankX_valid	Output	1	Read data valid
External/DMA Port			
ext_addr	Input	ADDR_WIDTH	External address
ext_bank_sel	Input	2	External bank select
ext_read	Input	1	External read enable
ext_write	Input	1	External write enable
ext_wdata	Input	DATA_WIDTH	External write data
ext_rdata	Output	DATA_WIDTH	External read data
ext_valid	Output	1	External read valid

Table 8: Signals of Tile Memory

- **Description:**

To support the high bandwidth required by the PEs, the global on-chip memory is partitioned into four independent banks (Row-Banked). This allows the DMA to stream data into one bank while the PEs simultaneously read from others. It acts as a customized scratchpad that bridges the gap between the slow external DRAM and the fast internal register files, ensuring that the PEs are constantly fed with active synaptic weights.

4.7. Array PE 4x4 (cgra_array_4x4.sv)

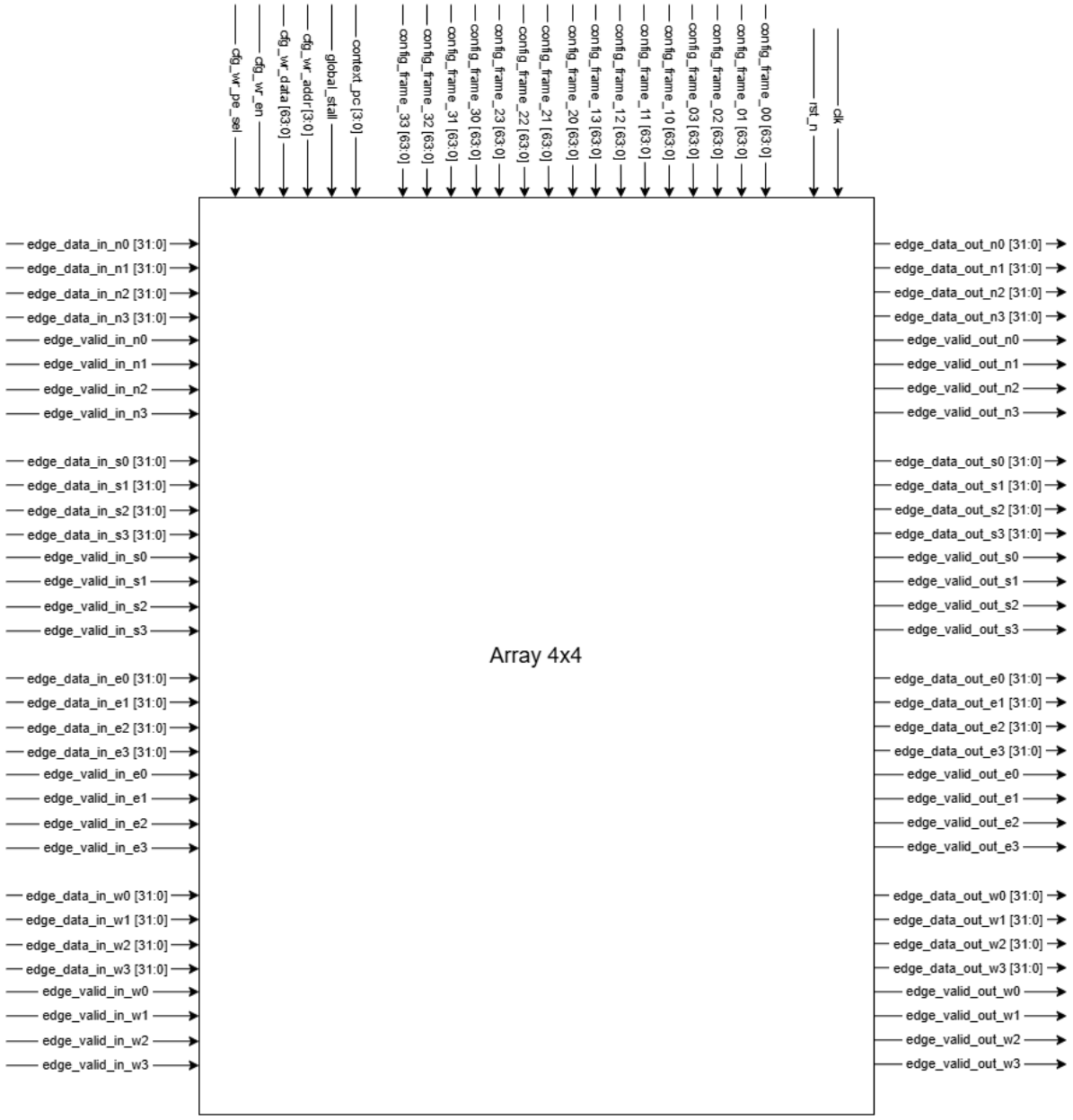

Figure 13: Module Array 4x4 PEs

Signal Name	Direction	Width	Description
clk, rst_n	Input	1	System Clock and Reset
Configuration			
config_frame_XX	Input	64	Static Config frame for PE (row,col) [00 to 33]
config_valid	Input	1	Config valid flag
context_pc	Input	PC_WIDTH	Context Program Counter
global_stall	Input	1	Stall signal

cfg_wr_addr	Input	PC_WIDTH	Config Write Address (Slot)
cfg_wr_data	Input	64	Config Write Data
cfg_wr_pe_sel	Input	4	Config Target PE Selection
cfg_wr_en	Input	1	Config Write Enable
Edge Inputs *(N=North, S=South, E=East, W=West)*			
edge_data_in_D#	Input	DATA_WIDTH	Data in from direction D, row/col #
edge_valid_in_D#	Input	1	Valid in from direction D, row/col #
Edge Outputs			
edge_data_out_D#	Output	DATA_WIDTH	Data out to direction D, row/col #
edge_valid_out_D#	Output	1	Valid out to direction D, row/col #

Table 9: Signals of Array 4x4

- **Description:**

This module instantiates the 4x4 grid of Processing Elements and defines the physical interconnects between them. It maps the 2D logical mesh topology into physical Verilog connections. The array supports multicast routing, where a single output from one PE can be broadcast to multiple neighbors simultaneously, mimicking the high fan-out connectivity of biological neural networks.

4.8. Module Title (cgra_tile.sv)

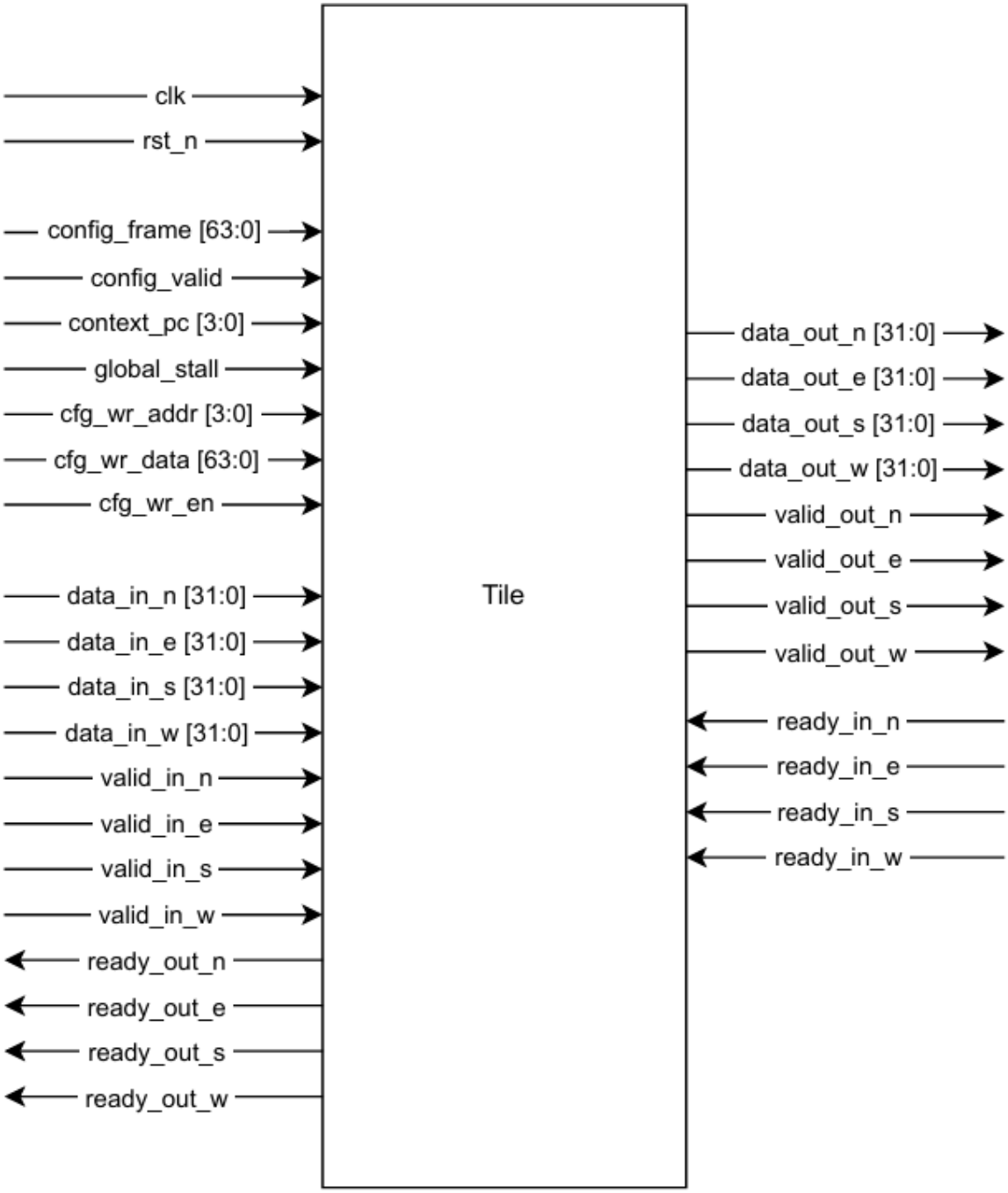

Figure 14: Module Tile

Signal Name	Direction	Width	Description
clk, rst_n	Input	1	Clock and Reset
config_frame	Input	64	Static configuration word
config_valid	Input	1	Configuration valid
context_pc	Input	PC_WIDTH	Context PC

global_stall	Input	1	Global Stall
cfg_wr_addr	Input	PC_WIDTH	Config Write Address
cfg_wr_data	Input	64	Config Write Data
cfg_wr_en	Input	1	Config Write Enable
Network Ports *(N, E, S, W)*			
data_in_X	Input	DATA_WIDTH	Data Input from direction X
valid_in_X	Input	1	Valid Input from direction X
ready_out_X	Output	1	Ready Output to direction X
data_out_X	Output	DATA_WIDTH	Data Output to direction X
valid_out_X	Output	1	Valid Output to direction X
ready_in_X	Input	1	Ready Input from direction X

Table 10: Signals of Tile

- **Description:**

The Tile module encapsulates a single PE and its associated Router. It standardizes the interface for tiling, making the array scalable. By modularizing the PE and Router together, the design allows for easier floorplanning and placement on the FPGA, ensuring that the critical path between the compute unit and the router is minimized.

4.9. Router (cgra_router.sv)

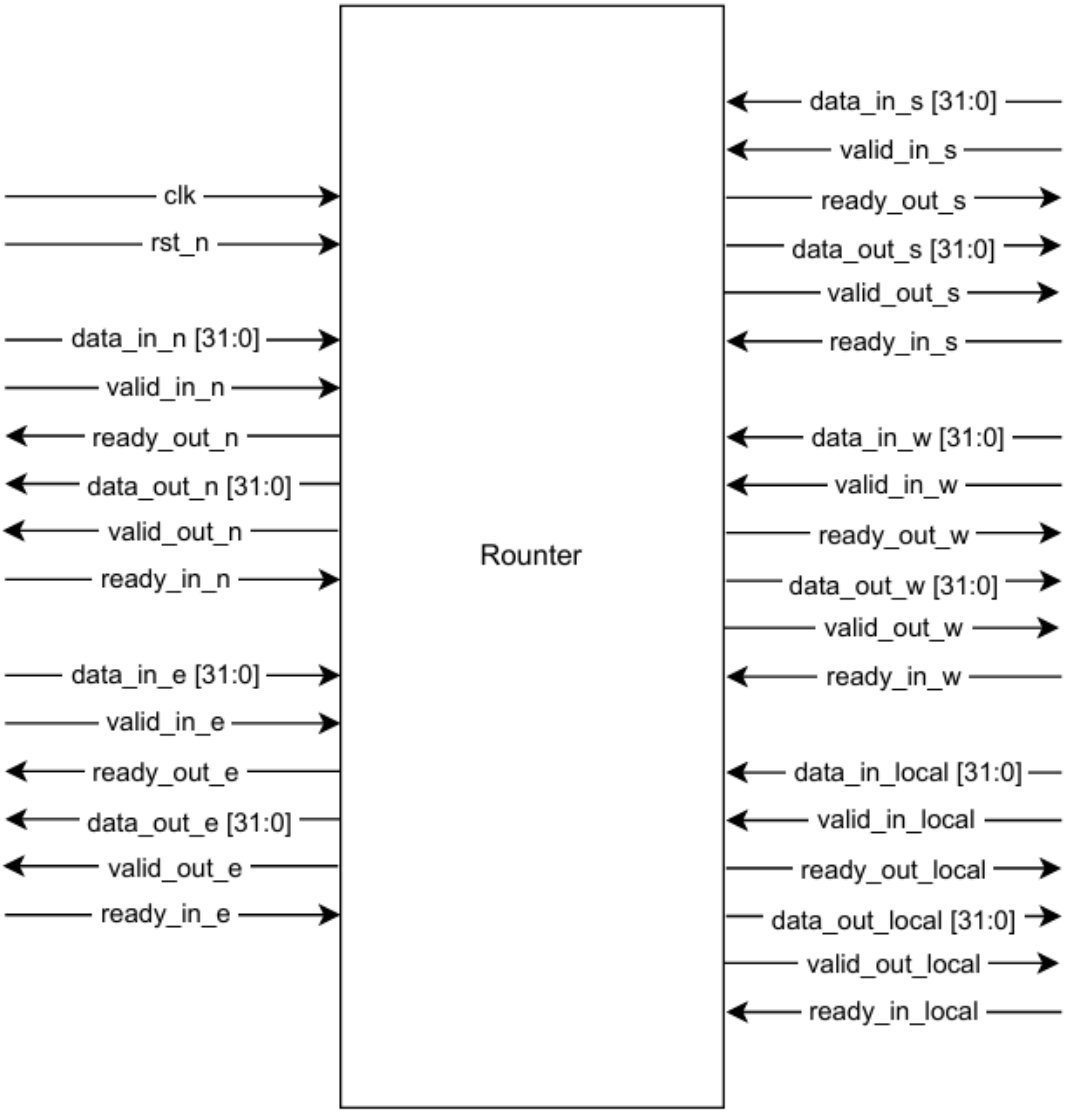

Figure 15: Module Router

Signal Name	Direction	Width	Description
clk, rst_n	Input	1	Clock and Reset
Ports N, E, S, W, Local *(Repeated for each port)*			
data_in_X	Input	DATA_WIDTH	Data Input
valid_in_X	Input	1	Valid Input
ready_out_X	Output	1	Ready Output
data_out_X	Output	DATA_WIDTH	Data Output
valid_out_X	Output	1	Valid Output
ready_in_X	Input	1	Ready Input

Table 11: Signals of Router

- **Description:**

The Router implements a 5-port (North, East, South, West, Local) mesh network. Unlike standard NoCs that use packet switching, this router uses circuit-switched, statically scheduled routing. The data movement is deterministic and defined by the configuration bits, eliminating the overhead of packet headers and arbitration logic. This is crucial for maintaining the cycle-accurate timing required for the CGRA's compiler execution model.

5. VERIFICATION AND TESTING

Ensuring the functional correctness of the CGRA accelerator is critical, given the complexity of the 4x4 mesh interconnect, the distributed LIF neuron updates, and the asynchronous DMA transactions. To guarantee production-quality, a robust, comprehensive, layered verification environment was developed using SystemVerilog and C++ co-simulation.

5.1. Verification Methodology

The verification strategy transitions from simple directed tests to a Constrained Random Verification (CRV) approach. This methodology is essential for uncovering corner-case bugs that manual testing often misses, such as pipeline hazards, bus contention, and arithmetic overflows.

The verification environment relies on a Self-Checking Architecture composed of three subsystems:

- Stimulus Generator (Driver): Produces randomized inputs (spike vectors, weights, configuration frames) constrained to valid ranges.
- Golden Reference Model: A high-level behavioral model that runs in parallel with the Design Under Test (DUT).

Scoreboard: A centralized checker that compares the DUT's output against the Golden Model's prediction in real-time.

5.2. Golden Model Architecture

The Golden Model serves as the source of truth for verification. Unlike the RTL (Register Transfer Level) code, which describes how the hardware is built (gates, flip-flops, wires), the Golden Model describes what the hardware should do (mathematical behavior).

- Behavioral Abstraction: The model computes results using 64-bit precision to detect overflows before casting to the 32-bit target width. For example, the saturation logic for arithmetic operations (like ADD and MAC) is implemented as follows:

```
case (op_idx)
    // ADD (Saturating)
    1: begin
        temp_sum = {{32{a_s[31]}}, a_s} + {{32{b_s[31]}}, b_s};
        if (temp_sum > 64'(MAX_POS)) expected = MAX_POS;
        else if (temp_sum < 64'(MIN_NEG)) expected = MIN_NEG;
        else expected = temp_sum[31:0];
    end

    // SUB (Saturating)
    2: begin
        temp_sum = {{32{a_s[31]}}, a_s} - {{32{b_s[31]}}, b_s};
        if (temp_sum > 64'(MAX_POS)) expected = MAX_POS;
        else if (temp_sum < 64'(MIN_NEG)) expected = MIN_NEG;
        else expected = temp_sum[31:0];
    end

    // MUL (Standard 32-bit truncation)
    3: expected = a * b;

    // MAC (Saturating + Accumulate)
    // For regression: Clear accumulator before each MAC test
    // to verify the multiplication/saturation part primarily.
    // NOTE: b is already sign-extended from 16-bit by the fix at line 65
    4: begin
        // MAC: Acc(0) + A * B (both signed, B is sign-extended 16-bit)
        temp_sum = (a_s * b_s);
        if (temp_sum > 64'(MAX_POS)) expected = MAX_POS;
        else if (temp_sum < 64'(MIN_NEG)) expected = MIN_NEG;
        else expected = temp_sum[31:0];
    end

    // Logic
    5: expected = a & b;
    6: expected = a | b;
    7: expected = a ^ b;
```

Figure 16: Golden Model Behavioral Logic

- Parallel Execution: For every transaction sent to the CGRA, the testbench sends the same inputs to this model. The model calculates the expected result instantly and compares it against the RTL output.
- Robustness: This specific logic ensures that even if an operation like 0x80000000 + 0xFFFFFFFF occurs (which would wrap around in standard

arithmetic), the model correctly predicts the saturated value MIN_NEG, forcing the hardware to match this safe behavior.

5.3. Testbench Components and Flow

The verification environment is structured as a layered testbench to maximize reuse and debuggability.

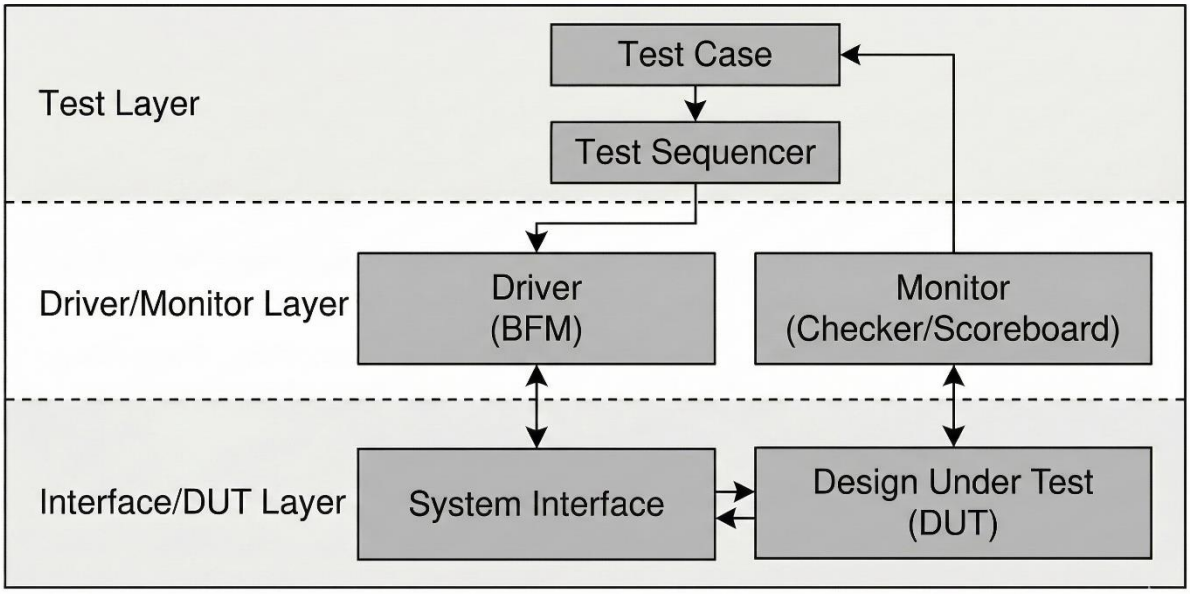

Figure 17: Layered Testbench Architecture

The Test Layer At the top of the hierarchy is the Test Layer, responsible for defining and orchestrating the verification scenarios.

- Test Case: This component initiates the simulation process. It holds the specific objectives of a given test run and determines pass/fail criteria based on feedback from lower layers.
- Test Sequencer: The Test Case passes high-level scenario requirements down to the Test Sequencer. The Sequencer decomposes these requirements into sequences of individual, abstract transactions to be executed.

The Driver/Monitor Layer The middle layer acts as a bridge between abstract transactions and concrete signal activity. It is split into stimulus generation (Driver) and observation (Monitor) paths.

- Driver (BFM): The Driver, often referred to as a Bus Functional Model (BFM), receives abstract transactions from the Test Sequencer. It is

responsible for converting these transactions into the specific pin-level signal protocols required to communicate with the hardware.

- Monitor (Checker/Scoreboard): Parallel to the driver, the Monitor passively observes the signals on the interface. It reconstructs signal activity back into abstract transactions. These observed transactions are typically passed to an internal or external Checker/Scoreboard to verify protocol compliance and data integrity against expected results.

The Interface/DUT Layer The bottom layer represents the physical connection to the hardware being verified.

- System Interface: This block represents the actual bundling of signals, clocks, and resets connecting the testbench to the design. The Driver actively drives signals onto this interface, while the Monitor passively samples signals from it.
- Design Under Test (DUT): This is the actual hardware logic being verified. It receives stimulus via the System Interface, processes it, and drives responses back onto the System Interface.

Execution Flow The overall data flow forms a closed loop:

1. The Test Case defines a scenario.
2. The Test Sequencer generates transaction sequences.
3. The Driver converts transactions into signal stimuli on the System Interface.
4. The DUT receives stimuli and drives responses back to the System Interface.
5. The Monitor observes these responses on the interface.
6. Results from the Monitor/Scoreboard are passed back up to the Test Case for final evaluation.

5.4. Verification Test Suites

The verification plan is executed through specialized test suites, totaling 6,010 test vectors.

- Suite A-F (Infrastructure): Validates the "plumbing" of the system—register read/writes, DMA boundary crossings, and interrupt generation.

- Suite AD (ISA Regression): The most rigorous suite. It runs 6,000 randomized vectors covering every Opcode in the ISA (ADD, SUB, MUL, LIF, etc.).
    - Randomization Strategy: Inputs are randomized to hit boundary cases, such as multiplying by zero, adding maximum positive integers (saturation testing), and triggering underflows with large negative numbers.
    - Coverage: Ensures 100% code coverage of the ALU and routing logic.

```
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_417: MAC(0xe0567ed6, 0x000079a7)
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_418: MAC(0x5a206d6d, 0xffffcacb)
[PERF] 9490M cycles @ 11297.6 kHz
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_419: MAC(0xc45eeb89, 0xffff9b93)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_420: MAC(0x80000000, 0xffffffff)
[PERF] 9500M cycles @ 11356.2 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_421: MAC(0x2d9d28c5, 0xffff99f5)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_422: MAC(0x34b33fa1, 0x0000394e)
[PERF] 9510M cycles @ 11333.1 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_423: MAC(0xd1fba59a, 0x000065e2)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_424: MAC(0xcb582267, 0xffffb35e)
[PERF] 9520M cycles @ 11356.1 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_425: MAC(0x9d7498b6, 0x00002eed)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_426: MAC(0x30402f08, 0x0000686f)
[PERF] 9530M cycles @ 11346.6 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_427: MAC(0x2e5f23da, 0xffffc2da)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_428: MAC(0x8774ce98, 0xffff8730)
[PERF] 9540M cycles @ 11352.9 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_429: MAC(0xc1879ca5, 0x00001670)
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_430: MAC(0x80000000, 0x00000001)
[PERF] 9550M cycles @ 11352.1 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_431: MAC(0xd7bce6f6, 0x00004e9a)
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_432: MAC(0x59c10a54, 0xffffaf04)
[PERF] 9560M cycles @ 11355.8 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_433: MAC(0x7991f6f5, 0xffffca3e)
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_434: MAC(0xc358ed1a, 0x00004ad8)
[PERF] 9570M cycles @ 11349.1 kHz
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_435: MAC(0xb70e740e, 0xffffd390)
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_436: MAC(0x25adbb53, 0xffff8ea5)
[PERF] 9580M cycles @ 11334 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_437: MAC(0x755231f5, 0xffffa4fe)
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_438: MAC(0x2186e412, 0xffffa9d1)
[PERF] 9590M cycles @ 11339.1 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_439: MAC(0x70032517, 0xffffd569)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_440: MAC(0x80000000, 0xffffffff)
[PERF] 9600M cycles @ 11339.2 kHz
  [DEBUG] pe_id=0, pe0_alu=0x80000000, actual=0x80000000, expected=0x80000000
  [PASS] AD04_441: MAC(0x24cd21cd, 0xffffeacb)
  [DEBUG] pe_id=0, pe0_alu=0x7fffffff, actual=0x7fffffff, expected=0x7fffffff
  [PASS] AD04_442: MAC(0xbbacc043, 0xffff867a)
```

Figure 18: Randomized ISA Regression log

5.5. Verification Results

The accelerator successfully passed the full regression suite with zero failures.

- Total Tests: 6010
- Pass rate: 100%
- Protocol Compliance: Zero AXI4/APB protocol violations detected by assertions.
- Status: The IP is verified robust and ready for synthesis and physical implementation.

```
[SUITE AD COMPLETE] 6000 Randomized tests finished.

╔═══════════════════════════════════════════════════════════╗
║                      COVERAGE REPORT                      ╒
╟───────────────────────────────────────────────────────────╢
  Transfer Types:
    Single-beat (4-16B):          0 hits
    Multi-beat (20-512B):         0 hits
    Max-burst (516-1024B):        0 hits
╟───────────────────────────────────────────────────────────╢
  Stress & Corner Cases:
    4KB Boundary Tests:           0 hits
    Stress Cycles:              100 hits
    Reset Recovery Tests:         0 hits
    Overlap Transactions:         0 hits
╚═══════════════════════════════════════════════════════════╝

╔═══════════════════════════════════════════════════════════╗
║                    PROTOCOL STATISTICS                    ╒
╟───────────────────────────────────────────────────────────╢
  AXI Transactions:
    AR (Read Address):      112021
    R  (Read Data):         112336
    AW (Write Address):        636
    W  (Write Data):           636
    B  (Write Response):       636
╟───────────────────────────────────────────────────────────╢
  Protocol Errors:                0
╚═══════════════════════════════════════════════════════════╝

================================================================
  FINAL RESULTS
================================================================
  PASSED: 6010
  FAILED: 0
  ASSERTIONS: 0 errors
  TOTAL:  6010
================================================================

  *** STATUS: PASSED (All Suites) - SILICON READY ***

- 01_bench/tb_top.sv:360: Verilog $finish

╔═══════════════════════════════════════════════════════════╗
║                  SIMULATION PERFORMANCE                   ╒
╟───────────────────────────────────────────────────────────╢
  Total sim cycles:    27453625001
  Wall clock time:     2420.99 sec
  Simulation speed:    11339.8 kHz
  Equiv real time:     2.74536e+07 us @ 100MHz
╚═══════════════════════════════════════════════════════════╝

[SIM] Simulation finished at time 27453625001

Waveform: 03_sim/cgra_sim.vcd
View with: gtkwave 03_sim/cgra_sim.vcd &
make[1]: Leaving directory '/__w/CGRA_project/CGRA_project'
```

Figure 19: Verification Results

6. IMPLEMENTATION RESULTS

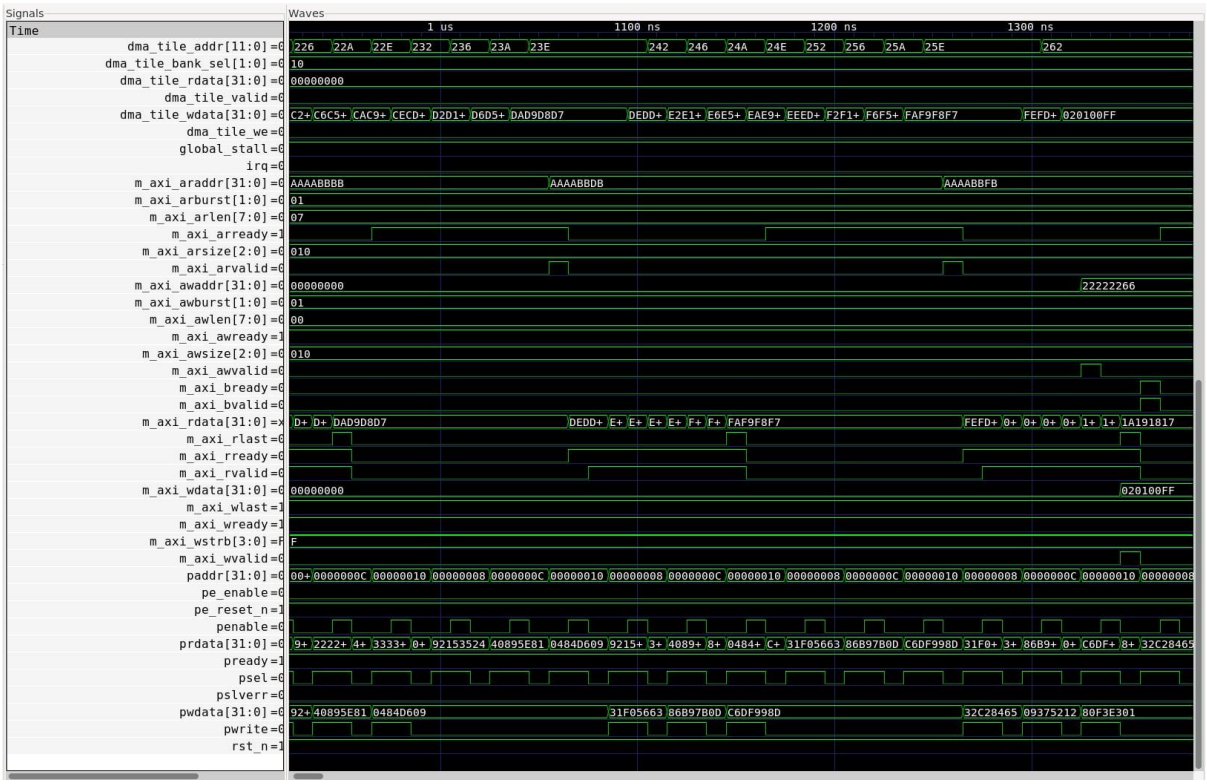

Figure 20: Waveform Test of CGRA IP

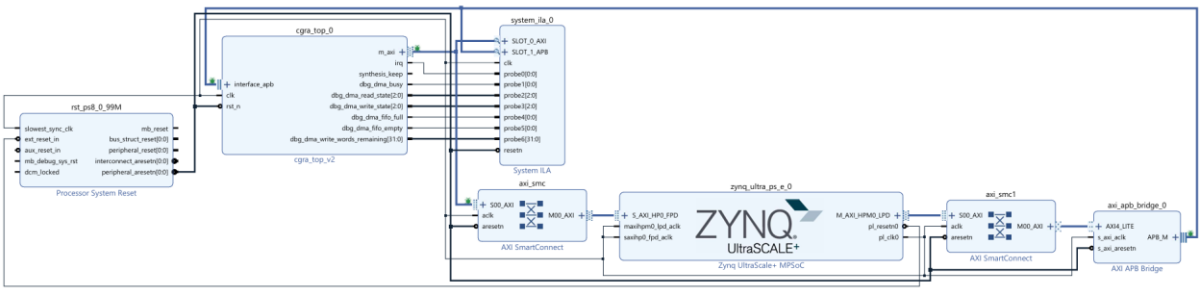

Figure 21: Block Design in Vivado

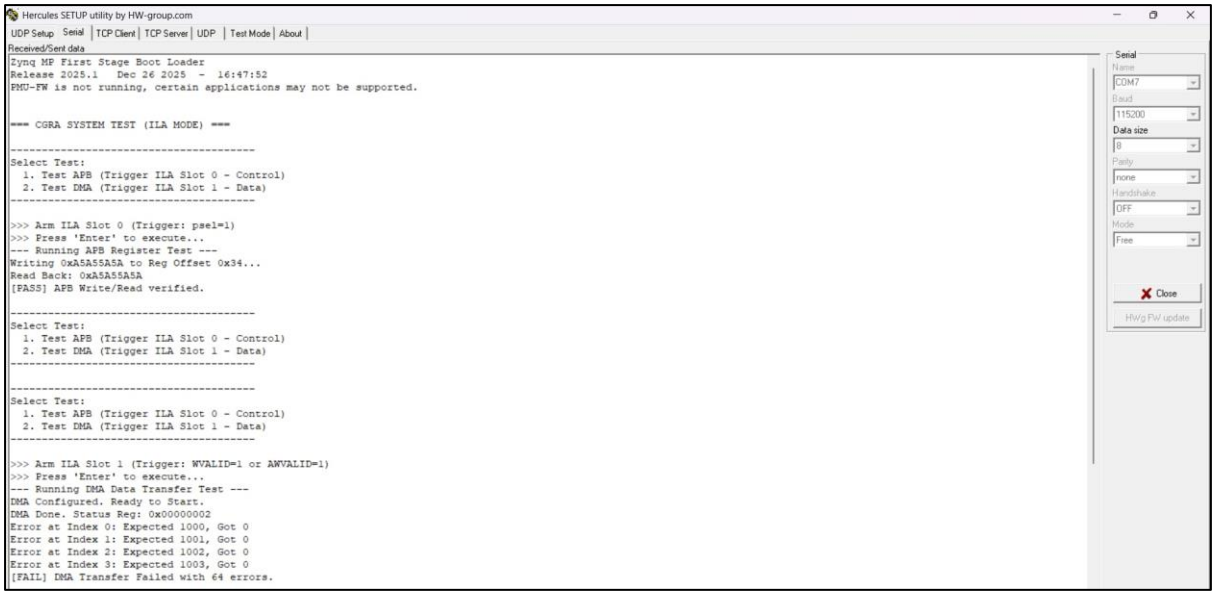

Figure 22: Result is shown on Hercules through UART

Utilization Post-Synthesis | **Post-Implementation**

Graph | **Table**

Resource	Utilization	Available	Utilization %
LUT	6093	87840	6.94
LUTRAM	1015	57600	1.76
FF	9045	175680	5.15
BRAM	9.50	128	7.42

Figure 23: Resource Utilization

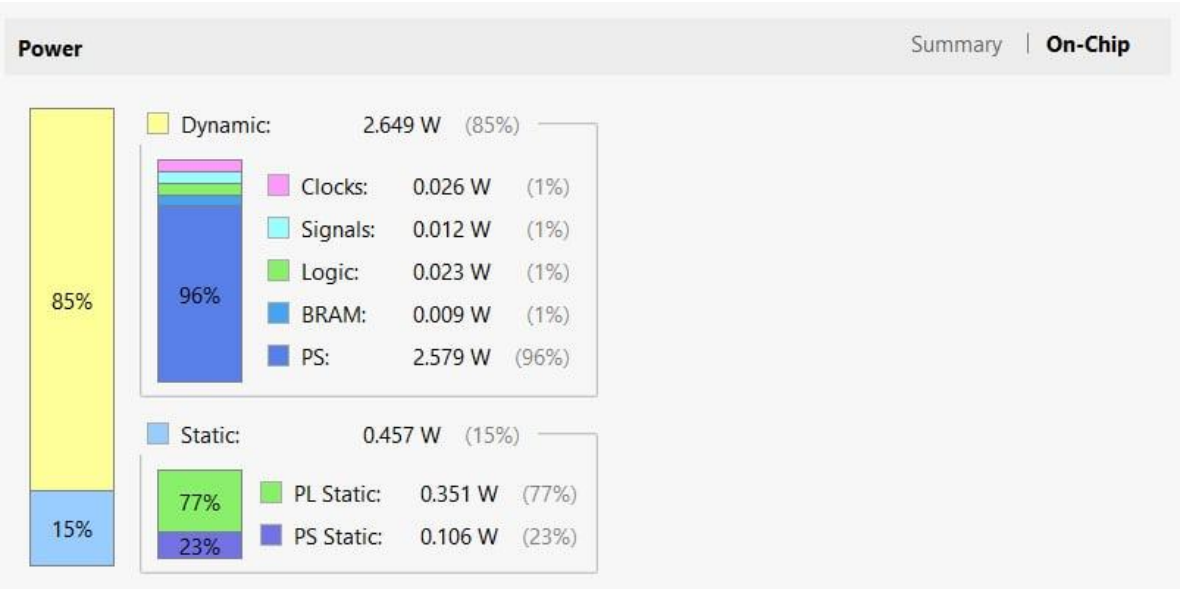

Figure 24: Power Report

Timing — **Setup** | Hold | Pulse Width

Worst Negative Slack (WNS):	5.321 ns
Total Negative Slack (TNS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	22708

Implemented Timing Report

Timing — Setup | **Hold** | Pulse Width

Worst Hold Slack (WHS):	0.012 ns
Total Hold Slack (THS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	22692

Implemented Timing Report

Timing — Setup | Hold | **Pulse Width**

Worst Pulse Width Slack (WPWS):	3.5 ns
Total Pulse Width Negative Slack (TPWS):	0 ns
Number of Failing Endpoints:	0
Total Number of Endpoints:	10889

Implemented Timing Report

Figure 25: Timing Report

- **Fmax Frequency Calculation:**

$$T = \frac{1}{100000000} = 10\ ns\ ; WNS = 5.321\ ns$$

$$F_{max}(MHz) = \max\left(\frac{1000}{T - WNS}\right) \quad (5.1)\ [14]$$

$$F_{max}(MHz) = \frac{1000}{(10-5.321)} = 213.72\ (MHz)$$

7. CONCLUSION AND DEVELOPMENT DIRECTION

7.1. Conclusion

This project successfully designed and implemented a **Coarse-Grained Reconfigurable Architecture (CGRA)** accelerator tailored for sparse Spiking Neural Network (SNN) inference. The architecture addresses the fundamental inefficiencies of von Neumann processors in handling irregular, event-driven workloads by integrating a 4x4 mesh of neuromorphic Processing Elements.

Key achievements include:

1. **Hardware-Software Integration:** A fully functional AXI4-based system that allows the ARM Cortex-A9 HPS to offload compute-intensive kernels to the FPGA fabric.
2. **SNN-Specific Features:** The integration of hardware LIF neuron updates and multicast routing directly accelerates the biological dynamics of spiking networks.
3. **Seamless Reconfiguration:** The double-buffered configuration memory enables "hot-swapping" of layers, hiding reconfiguration latency and maintaining high PE utilization.
4. **Robust Verification:** With 100% pass rate on 6010 test vectors, the IP is robust and ready for physical deployment.

7.2. Development Direction

Although the proposed CGRA accelerator has demonstrated correct functionality and promising architectural advantages for sparse Spiking Neural Network (SNN) inference, several extensions and improvements can be explored in future work to further enhance its performance, flexibility, and applicability.

First, the current implementation focuses primarily on inference workloads with statically configured weights and neuron parameters. A natural extension is to support on-chip learning mechanisms, such as Spike-Timing-Dependent Plasticity (STDP) or other local learning rules. Integrating programmable learning engines

into the Processing Elements (PEs) would enable online adaptation and move the architecture closer to fully neuromorphic systems.

Second, the scalability of the CGRA can be further investigated. While the present design employs a fixed 4×4 mesh of PEs, future versions could explore parameterizable array sizes and hierarchical interconnects to support larger neural layers. Additionally, adaptive routing strategies or hybrid static–dynamic routing could be evaluated to improve performance for irregular or large-scale sparse workloads.

Third, the current accelerator uses fixed-point arithmetic optimized for efficiency and determinism. Future work may consider mixed-precision or configurable precision support, allowing different layers of an SNN to trade accuracy for energy efficiency. This would be particularly beneficial for deep or multi-layer SNN models deployed in edge or low-power environments.

Another important direction is software and compiler support. Developing a higher-level compiler or mapping tool that automatically translates trained SNN models (e.g., from PyTorch-based frameworks) into CGRA configuration bitstreams and memory layouts would significantly improve usability and reduce development effort. Such a toolchain could also perform automated scheduling and placement optimizations tailored to the CGRA architecture.

Finally, comprehensive performance and energy evaluation on real-world SNN benchmarks remains an important future task. Comparing the proposed CGRA against CPUs, GPUs, and other neuromorphic accelerators using standardized datasets and metrics would provide deeper insights into its practical advantages and limitations. Power measurements on hardware would further validate the suitability of the architecture for energy-constrained edge computing applications.

In summary, these development directions highlight multiple opportunities to extend the proposed CGRA accelerator into a more scalable, programmable, and intelligent neuromorphic computing platform.

REFERENCES

[1] A. Podobas, K. Sano, and S. Matsuoka, "A Survey on Coarse-Grained Reconfigurable Architectures from a Performance Perspective," *arXiv preprint arXiv:2004.04509*, 2020. arXiv

[2] L. Liu *et al.*, "A Survey of Coarse-Grained Reconfigurable Architecture," *ACM Computing Surveys*, 2019. ACM Digital Library

[3] W. Maass, "Networks of Spiking Neurons: The Third Generation of Neural Network Models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997. ScienceDirect

[4] W. Gerstner and W. M. Kistler, *Spiking Neuron Models: Single Neurons, Populations, Plasticity*. Cambridge University Press, 2002. EPFL+1

[5] M. Pfeiffer and T. Pfeil, "Deep Learning With Spiking Neurons: Opportunities and Challenges," *Frontiers in Neuroscience*, vol. 12, 2018, Art. no. 774. Frontiers

[6] K. Roy, A. Jaiswal, and P. Panda, "Towards Spike-Based Machine Intelligence with Neuromorphic Computing," *Nature*, vol. 575, pp. 607–617, 2019. nature.com

[7] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate Gradient Learning in Spiking Neural Networks," *arXiv preprint arXiv:1901.09948*, 2019. arXiv

[8] B. Rueckauer, I.-A. Lungu, Y. Hu, and S.-C. Liu, "Conversion of Continuous-Valued Deep Networks to Efficient Event-Driven Networks for Image Classification," *Frontiers in Neuroscience*, vol. 11, 2017, Art. no. 682. Frontiers

[9] P. U. Diehl, D. Neil, J. Binas, M. Cook, S.-C. Liu, and M. Pfeiffer, "Fast-Classifying, High-Accuracy Spiking Deep Networks Through Weight and Threshold Balancing," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2015. ResearchGate

[10] G.-Q. Bi and M.-M. Poo, "Synaptic Modifications in Cultured Hippocampal Neurons: Dependence on Spike Timing, Synaptic Strength, and Postsynaptic Cell

Type," *The Journal of Neuroscience*, vol. 18, no. 24, pp. 10464–10472, 1998. PubMed

[11] W. A. Wulf and S. A. McKee, "Hitting the Memory Wall: Implications of the Obvious," Univ. of Virginia, Tech. Rep., 1995. Libra Open+1

[12] S. W. Williams, N. Bell, J. W. Choi, and R. Vuduc, "Sparse Matrix-Vector Multiplication on Multicore and Accelerator Systems," (chapter/online version), 2010. NVIDIA+1

[13] Arm, "Introduction to AMBA AXI4," *Arm Developer*, Issue 01, 2020. Arm Developer+1

[14] AMD, "Assessing the Maximum Frequency of the Design," *UltraFast Design Methodology Guide for FPGAs and SoCs (UG949)*, 2025. Website: AMD Docs